

# Redes Neuronales Artificiales - Parte Cuatro

## Técnicas de Inteligencia Artificial

Prof. Flavio Prieto  
email: faprieto@unal.edu.co

INGENIERÍA MECATRÓNICA  
Facultad de Ingeniería  
Universidad Nacional de Colombia Sede Bogotá



5 de marzo de 2026

En esta sección se abordarán los siguientes temas:

1. **Aplicaciones de las Redes Neuronales.**
2. **Normalización de Datos.**
3. **Técnicas de Entrenamiento en Machine Learning.**
4. **Métricas de Evaluación en Machine Learning.**
5. **Desbalance de Datos.**

## Aplicaciones de las Redes Neuronales.

Las redes neuronales son modelos computacionales versátiles que se aplican en una amplia gama de problemas reales, gracias a su capacidad para aproximar funciones no lineales, clasificar patrones complejos y aprender representaciones jerárquicas de los datos.

| Área                  | Función típica                                        |
|-----------------------|-------------------------------------------------------|
| Series temporales     | Predicción, regresión, clasificación de patrones      |
| Audio y voz           | Clasificación, predicción, generación, interpolación  |
| Visión por computador | Clasificación, segmentación, detección, interpolación |
| NLP                   | Clasificación, predicción de secuencias, generación   |

## Predicción en series temporales.

- ▶ La predicción en series temporales consiste en estimar valores futuros de una variable basándose en su comportamiento pasado:
  - ▶ Se utilizan datos secuenciales ordenados en el tiempo.
  - ▶ El objetivo principal es **anticipar** el valor siguiente o varios valores futuros.
- ▶ Este tipo de predicción es clave en áreas como finanzas, meteorología, demanda energética o mantenimiento predictivo.

## Tipos comunes de modelos para series temporales.

- ▶ **Modelos autoregresivos (AR):**

$$y_t = w_0 + w_1 y_{t-1} + \dots + w_p y_{t-p} + \varepsilon_t$$

Predicen usando valores pasados de la misma serie.

- ▶ **Modelos con medias móviles (MA):**

$$y_t = w_0 + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}$$

Consideran errores pasados.

- ▶ **Modelos ARIMA:** combinan componentes AR y MA para capturar tendencias y estacionalidad.
- ▶ **Modelos multivariantes:** usan varias series temporales como entradas:

$$\hat{y}_t = f(y_{t-1}, \dots, y_{t-p}, x_{t-1}, \dots, x_{t-p})$$

## Redes neuronales.

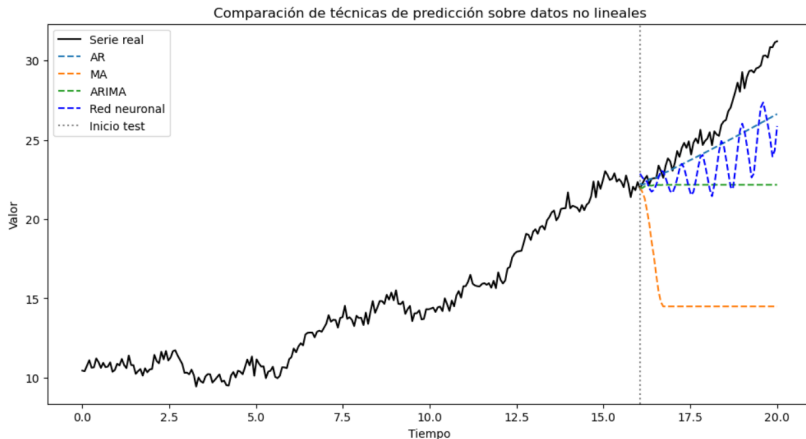
- ▶ Las redes neuronales pueden aprender relaciones no lineales complejas y dependencias de largo plazo en los datos.
- ▶ Se usan distintas arquitecturas:
  - ▶ Redes feedforward simples (MLP).
  - ▶ Redes recurrentes (RNN) y variantes como LSTM o GRU.
  - ▶ Redes convolucionales (CNN) para extraer patrones locales.
- ▶ La red aprende una función:

$$\hat{y}_{t+1} = f(y_t, y_{t-1}, \dots, y_{t-p}; \theta)$$

donde:

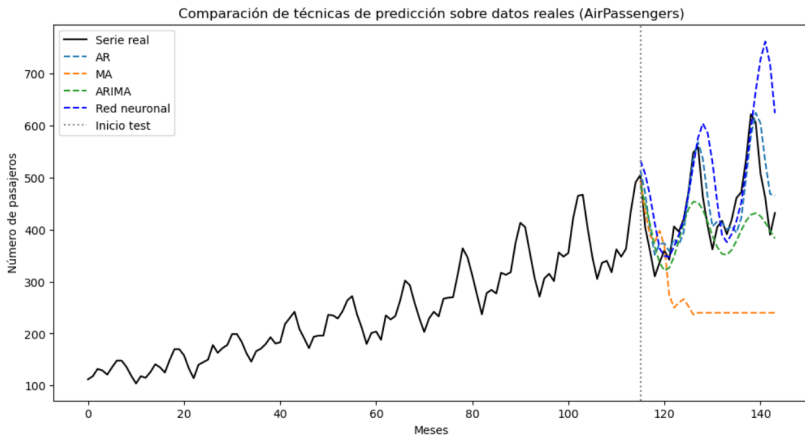
- ▶  $y_t, y_{t-1}, \dots$ : valores anteriores de la serie.
- ▶  $\theta$ : parámetros aprendidos.
- ▶  $f$ : mapeo aprendido por la red.
- ▶ La red se entrena minimizando una función de pérdida, por ejemplo, el error cuadrático medio (MSE).

## Ejemplo.





## Ejemplo: Predicción del número mensual de pasajeros de aerolíneas internacionales.



**Ver código python.**

## Regresión.

- ▶ La regresión es un método estadístico que permite modelar la relación entre:
  - ▶ Una variable dependiente (también llamada variable de respuesta o salida).
  - ▶ Una o más variables independientes (también llamadas predictores o características).
- ▶ El objetivo principal es **predecir** o **estimar** el valor de la variable dependiente a partir de los valores conocidos de las variables independientes.

## Tipos comunes de regresión.

- ▶ **Regresión lineal:** ajusta una recta

$$y = w_0 + w_1x$$

donde  $w_0$  es el intercepto y  $w_1$  la pendiente.

- ▶ **Regresión polinómica:** utiliza un polinomio de grado  $p$

$$y = w_0 + w_1x + w_2x^2 + \dots + w_px^p$$

para capturar relaciones más complejas.

- ▶ **Regresión múltiple:** considera varias variables predictoras

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_kx_k$$

donde  $x_1, x_2, \dots, x_k$  son las características.

- ▶ **Regresión no lineal:** ajusta modelos donde la relación no es lineal ni polinómica

$$y = f(x; \theta)$$

siendo  $f$  una función no lineal parametrizada.

## Redes neuronales en regresión.

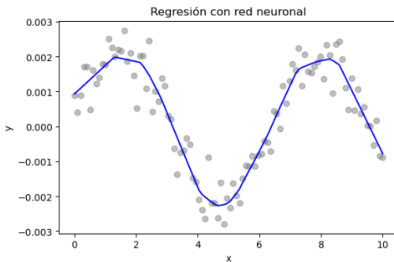
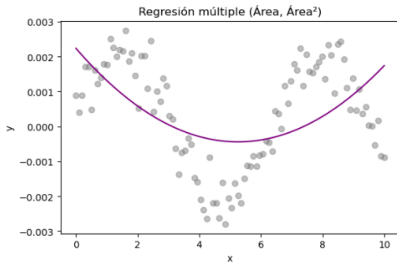
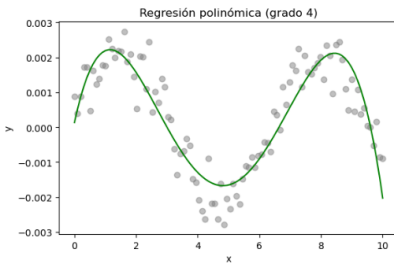
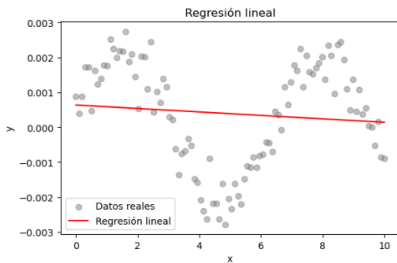
- ▶ Las redes neuronales pueden usarse como modelos de regresión para **predecir valores continuos**.
- ▶ A diferencia de modelos lineales clásicos, las redes **permiten aprender funciones no lineales complejas** gracias a las capas ocultas y funciones de activación.
- ▶ En el caso de regresión, **típicamente se usa una capa de salida con una única neurona (o varias si hay varias variables objetivo)** y función de activación lineal:

$$\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$$

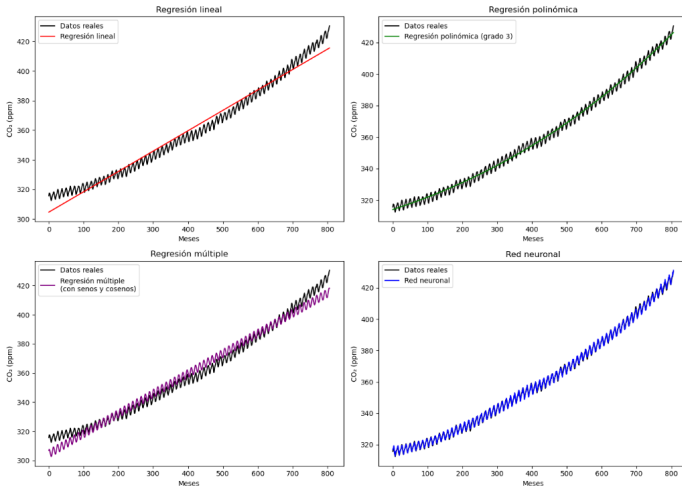
donde:

- ▶  $\mathbf{x}$ : vector de entrada.
- ▶  $\boldsymbol{\theta}$ : pesos y sesgos aprendidos.
- ▶  $f$ : función representada por la red.
- ▶ La red se entrena minimizando una función de pérdida, por ejemplo, el error cuadrático medio (MSE).

## Ejemplo.



## Ejemplo: Concentración de CO<sub>2</sub> en la atmósfera en Mauna Loa (Hawái).



**Ver código python.**

## Clasificación.

- ▶ La clasificación es una tarea supervisada que consiste en asignar una etiqueta o clase a cada ejemplo de entrada, según sus características:
  - ▶ Cada ejemplo pertenece a una de varias categorías previamente definidas.
  - ▶ Se construyen modelos capaces de aprender patrones y relaciones para distinguir estas clases.
- ▶ El objetivo principal es **predecir correctamente** la clase a la que pertenece un nuevo ejemplo no visto.

## Tipos comunes de clasificación.

- ▶ **Clasificación binaria:** dos posibles clases.

$$y \in \{0, 1\}$$

Ejemplo: detectar spam o no spam.

- ▶ **Clasificación multiclase:** más de dos clases mutuamente excluyentes.

$$y \in \{1, 2, \dots, K\}$$

Ejemplo: reconocer dígitos del 0 al 9.

- ▶ **Clasificación multietiqueta:** cada ejemplo puede pertenecer a varias clases simultáneamente.

$$\mathbf{y} \in \{0, 1\}^K$$

Ejemplo: asignar varios temas a un documento.



## Redes neuronales en clasificación.

- ▶ Las redes neuronales permiten aprender funciones de decisión complejas para separar clases en espacios de alta dimensión.
- ▶ Normalmente se usa una capa de salida con:
  - ▶ Una neurona con función sigmoide para clasificación binaria.
  - ▶ Varias neuronas con softmax para clasificación multiclase.
- ▶ La red aprende un mapeo:

$$\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$$

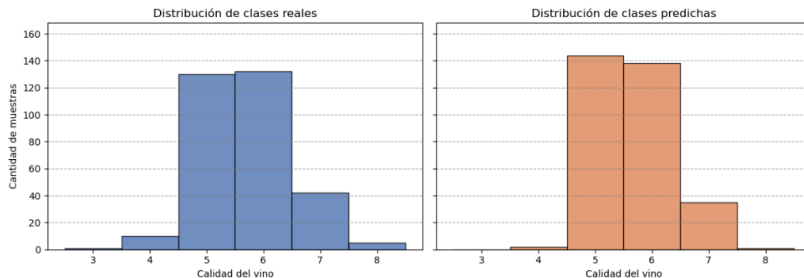
donde:

- ▶  $\mathbf{x}$ : vector de entrada.
- ▶  $\boldsymbol{\theta}$ : pesos y sesgos aprendidos.
- ▶  $f$ : función representada por la red.
- ▶ Se entrena minimizando funciones de pérdida como:
  - ▶ Entropía cruzada para clasificación binaria o multiclase.

## Ejemplo. Clasificación multiclase de la calidad del vino.

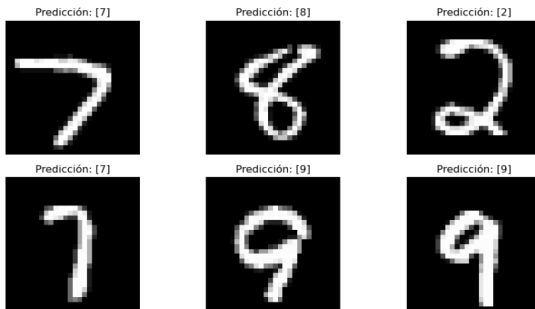
- ▶ Muestra 0: calidad real=6, predicha=5
- ▶ Muestra 258: calidad real=5, predicha=6
- ▶ Muestra 47: calidad real=6, predicha=6
- ▶ Muestra 86: calidad real=5, predicha=5
- ▶ Muestra 132: calidad real=5, predicha=5

Comparación de distribución de clases reales vs. predichas



## Ejemplo: Clasificación de imágenes de dígitos manuscritos (MNIST<sup>1</sup>).

Ejemplos clasificados por el modelo



**Ver código python.**

<sup>1</sup>La base de datos MNIST de dígitos escritos a mano tiene un conjunto de entrenamiento de 60.000 ejemplos y un conjunto de prueba de 10.000 ejemplos.

## Generación de datos.

- ▶ La generación consiste en crear datos sintéticos que imitan el comportamiento o las características de datos reales:
  - ▶ Puede aplicarse a imágenes, texto, audio, series temporales, entre otros.
  - ▶ Permite crear contenido nuevo, aumentar datasets o realizar tareas creativas.
- ▶ En el caso del audio, por ejemplo, se pueden generar voces sintéticas, efectos sonoros o incluso música.

## Tipos comunes de generación.

- ▶ **Modelos autoregresivos:** generan cada elemento condicionado en los anteriores.

$$x_t = f(x_{t-1}, x_{t-2}, \dots; \theta)$$

Ejemplo: síntesis de voz por WaveNet.

- ▶ **Redes generativas adversarias (GANs):** aprenden a generar datos que imitan la distribución real.

$$G(z) \approx x$$

donde  $G$  es el generador y  $z$  un vector aleatorio.

- ▶ **Modelos basados en transformadores:** generan secuencias completas aprovechando mecanismos de atención.
- ▶ **Modelos auto-regresivos recurrentes:** útiles para generar audio continuo, nota por nota o frame por frame.

## Redes neuronales en generación de datos.

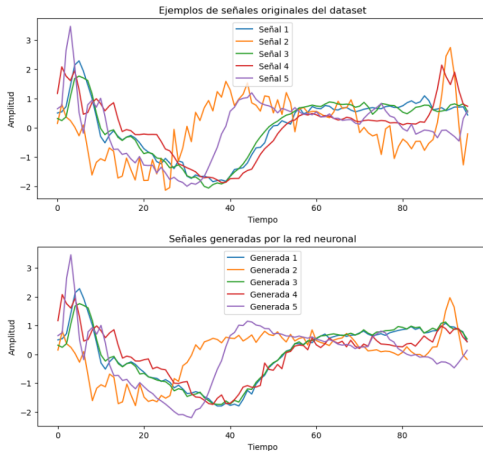
- ▶ Las redes neuronales pueden aprender distribuciones complejas y generar nuevos datos similares a los de entrenamiento.
- ▶ Ejemplos conocidos:
  - ▶ **GANs (Generative Adversarial Networks)**: generan imágenes, texto, audio o incluso datos sintéticos para aumentar conjuntos de datos.
  - ▶ **Modelos autoregresivos**: generan secuencias, como texto, series temporales o música, aprendiendo dependencias entre pasos sucesivos.
- ▶ El modelo aprende una función:

$$\hat{x}_t = f(x_{<t}; \theta)$$

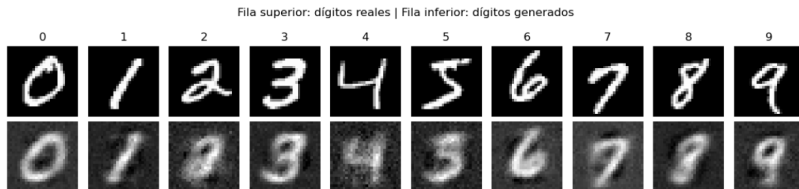
donde:

- ▶  $x_{<t}$ : datos previos o contexto, y  $\theta$ : parámetros de la red.
- ▶ La calidad de los datos generados depende de la arquitectura, la cantidad y diversidad de datos de entrenamiento, así como de la función de pérdida utilizada.

## Ejemplo. señales reales de electrocardiograma (ECG).



## Ejemplo: Generación de dígitos manuscritos.



**Ver código python.**



## Interpolación.

- ▶ La interpolación busca construir una **función continua** que pase exactamente por un conjunto de puntos conocidos:
  - ▶ Se parte de datos de entrada  $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ .
  - ▶ Se construye una función  $f$  que cumpla:

$$f(x_i) = y_i \quad \forall i = 1, \dots, N.$$

- ▶ El objetivo es estimar valores intermedios  $f(x^*)$  para  $x^*$  que no estaban originalmente en el conjunto de entrenamiento.

## Tipos comunes de interpolación.

- ▶ **Interpolación lineal:** entre dos puntos, usa una recta:

$$f(x) = y_0 + (y_1 - y_0) \frac{x - x_0}{x_1 - x_0}.$$

- ▶ **Interpolación polinómica:** ajusta un único polinomio de grado  $N - 1$  que pasa por todos los puntos:

$$f(x) = a_0 + a_1x + \cdots + a_{N-1}x^{N-1}.$$

- ▶ **Splines:** usa piezas de polinomios de bajo grado que se conectan suavemente.
- ▶ **Interpolación basada en redes neuronales:** la red aprende a reproducir exactamente los valores conocidos y estimar valores intermedios de forma suave.

## Redes neuronales en interpolación.

- ▶ Las redes neuronales pueden actuar como interpoladores universales: se entrenan para reproducir los puntos conocidos.
- ▶ La red aprende una función  $f(\mathbf{x}; \boldsymbol{\theta})$  que cumple:

$$f(x_i; \boldsymbol{\theta}) \approx y_i.$$

- ▶ Gracias a su capacidad de aproximación universal, la red no solo pasa por los datos conocidos, sino que también suaviza la transición entre ellos.
- ▶ Suele usarse una capa de salida con activación lineal, de forma similar a la regresión.

## Ejemplo de interpolación.

- Suponga cuatro puntos conocidos:

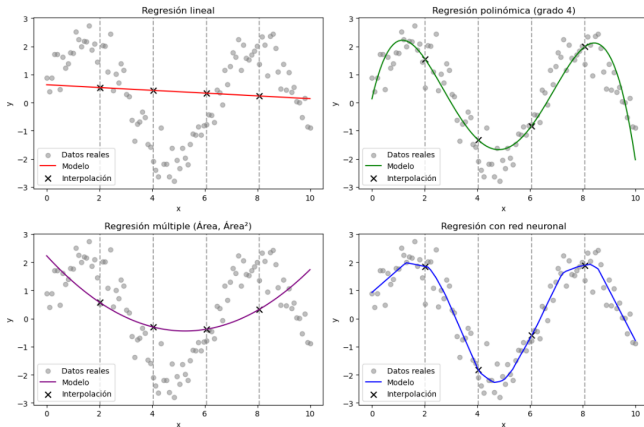
$$(1, 2), (2, 3), (3, 2.5), (4, 4)$$

- La red neuronal se entrena para que:

$$f(1) = 2, \quad f(2) = 3, \quad f(3) = 2.5, \quad f(4) = 4.$$

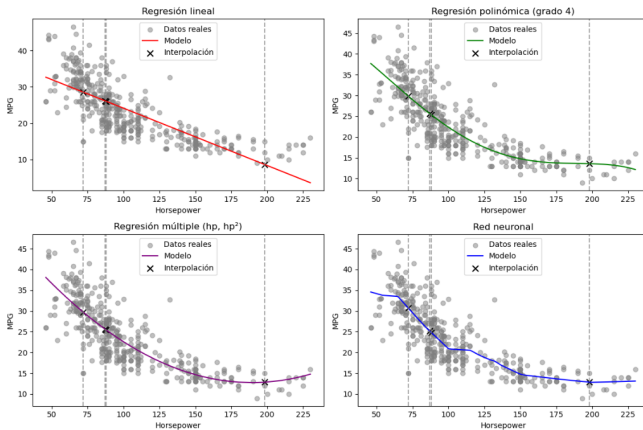
- Luego predice suavemente valores para  $x = 2.5, 3.5$ , etc.

## Visualización gráfica.



- Se observa cómo la curva aprendida pasa por los puntos conocidos, suavizando la forma entre ellos.

**Ejemplo.** interpolar y ajustar varios modelos para aproximar la relación entre la potencia del motor y el consumo de combustible, y comparar sus resultados.



**Ver código python.**

## Segmentación.

- ▶ La segmentación es la tarea de dividir un dato (como una imagen, un audio o un texto) en partes o regiones significativas:
  - ▶ En imágenes, se busca identificar y separar objetos, bordes o regiones.
  - ▶ En audio o texto, se pueden segmentar eventos, palabras o frases.
- ▶ En visión por computador, permite localizar y delimitar áreas de interés como carreteras, personas o defectos.

## Tipos comunes de segmentación.

- ▶ **Segmentación semántica:**

Clasifica cada píxel  $\rightarrow$  clase correspondiente

Ejemplo: cada píxel como carretera, vehículo o peatón.

- ▶ **Segmentación por instancia:**

Distingue objetos individuales de la misma clase

Ejemplo: identificar dos autos distintos aunque ambos sean “auto”.

- ▶ **Segmentación panóptica:** Combina semántica e instancia para una representación más completa.
- ▶ **Segmentación en otras señales:** En audio: detectar inicio y fin de palabras. En texto: segmentar oraciones, párrafos o entidades.



## Redes neuronales en segmentación.

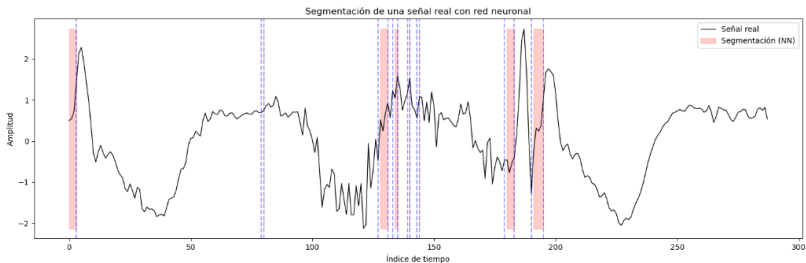
- ▶ Las redes aprenden a asignar etiquetas a cada elemento (píxel, frame o carácter).
- ▶ Arquitecturas comunes:
  - ▶ **Fully Convolutional Networks (FCN)**: generan mapas de segmentación del mismo tamaño que la entrada.
  - ▶ **U-Net**: muy usada en imágenes médicas; combina codificador y decodificador con “skip connections”.
- ▶ Formalmente, aprenden una función:

$$\hat{y} = f(x; \theta)$$

donde:

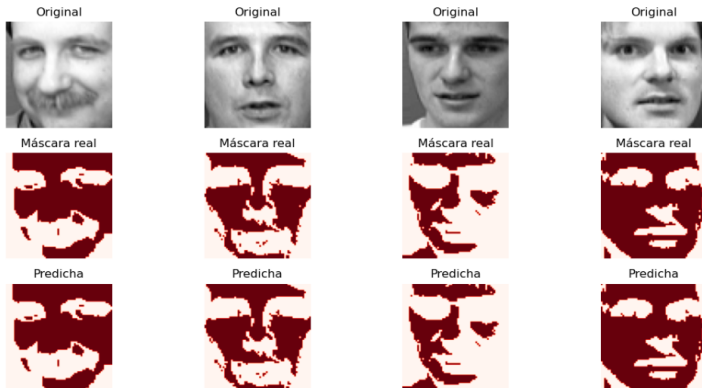
- ▶  $x$ : entrada (imagen, audio, texto).
- ▶  $\hat{y}$ : máscara o etiquetas por elemento.
- ▶  $\theta$ : parámetros de la red.
- ▶ Se entrenan minimizando funciones de pérdida como entropía cruzada o loU.

**Ejemplo.** Identificar puntos de la señal que superen un umbral ('media + desviación estándar'), considerándolos como segmentos relevantes".



**Ejemplo:** Segmentar las regiones más brillantes (intensidad mayor que la media global).

Segmentación de imágenes reales (caras) con red neuronal



[Ver código python.](#)

## Detección.

- ▶ La detección consiste en localizar y reconocer la presencia de uno o varios objetos o eventos dentro de una señal o imagen:
  - ▶ En imágenes: dibujar recuadros alrededor de objetos (autos, peatones, señales).
  - ▶ En audio: identificar instantes donde ocurre un sonido concreto.
  - ▶ En texto: detectar palabras clave, nombres propios o entidades.
- ▶ Permite tareas como conducción autónoma, análisis de vídeos, control de calidad o reconocimiento de acciones.

## Tipos comunes de detección.

- ▶ **Detección de un solo objeto:**

Localiza y clasifica el objeto más relevante de la imagen.

- ▶ **Detección de múltiples objetos:**

Encuentra y clasifica todos los objetos presentes.

- ▶ **Detección en tiempo real:**

Modelos diseñados para baja latencia, como YOLO o SSD.

- ▶ **Detección de eventos en señales:**

En audio: detectar eventos acústicos como aplausos o sirenas.

En texto: detectar entidades o patrones específicos.

## Redes neuronales en detección.

- ▶ Las redes neuronales detectan objetos aprendiendo simultáneamente:
  - ▶ La clase del objeto.
  - ▶ Su localización (generalmente coordenadas de un recuadro).
- ▶ Arquitecturas populares:
  - ▶ **R-CNN** (*Region-based Convolutional Neural Network*) y variantes: dos etapas, primero proponen regiones, luego clasifican.
  - ▶ **YOLO, SSD**: detección en una sola etapa, más rápidas.
- ▶ La red aprende una función:

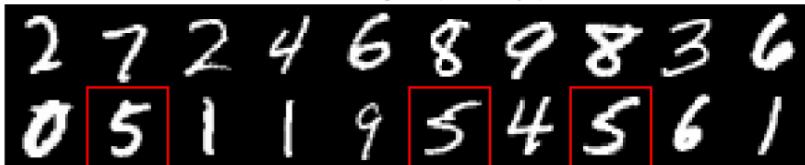
$$\hat{y} = f(x; \theta)$$

donde:

- ▶  $x$ : imagen o secuencia de entrada.
- ▶  $\hat{y}$ : conjunto de recuadros y clases.
- ▶  $\theta$ : parámetros de la red.
- ▶ Se entrenan minimizando pérdidas combinadas de localización (por ejemplo IoU o L1) y clasificación (entropía cruzada, etc.).

**Ejemplo.** Detección del dígito 5.

Detección de dígito '5' (cuadros rojos)

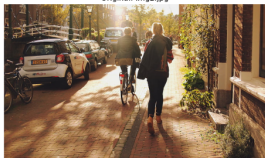


## Ejemplo. Detección de peatones y autos usando Yolo.

Procesando: ./fig/Notebook/Yolo\Img1.jpg  
Original: Img1.jpg



Procesando: ./fig/Notebook/Yolo\Img2.jpg  
Original: Img2.jpg



**Ver código python.**



## Normalización de Datos en Machine Learning.

- ▶ En ML, los datos pueden tener escalas muy diferentes (p.e. edad en años vs. ingresos en millones).
- ▶ **Problema:** Los algoritmos sensibles a la magnitud pueden verse dominados por las características con valores mayores.
- ▶ **Normalización** ayuda a:
  - ▶ Evitar que una variable influya desproporcionadamente.
  - ▶ Mejorar la **velocidad de convergencia** durante el entrenamiento.
  - ▶ Hacer que las distancias (usadas en k-NN, clustering) sean más representativas.
- ▶ Es esencial en redes neuronales y algoritmos basados en gradientes.
- ▶ También facilita la visualización y análisis de datos.

## Normalización de Datos

En machine learning, la normalización busca que las variables tengan una escala o distribución más adecuada para el entrenamiento de modelos.

Existen dos grandes familias:

- ▶ **Técnicas de preprocesamiento de datos:**  
se aplican antes del entrenamiento, sobre los datos de entrada.
- ▶ **Técnicas de normalización interna:**  
se aplican dentro del modelo, durante el entrenamiento.

## Técnicas de preprocesamiento de datos.

- ▶ Estas técnicas buscan **ajustar la escala de los datos de entrada**, sin alterar su estructura relativa ni la información esencial.
- ▶ También buscan asegurar que las distintas **características (features) estén en rangos comparables**, evitando que alguna domine el aprendizaje debido a su magnitud.
- ▶ Ayudan a **mejorar la estabilidad numérica**, facilitar la **convergencia** de algoritmos de optimización y cumplir supuestos estadísticos de ciertos modelos (como SVMs, PCA, regresión logística).

## Normalización Min-Max

- ▶ La técnica de normalización Min-Max transforma los datos para que se ajusten a un rango determinado, usualmente  $[0, 1]$  o  $[-1, 1]$ .
- ▶ Esta técnica **mantiene la distribución original de los datos, pero reescala sus valores**, permitiendo comparabilidad entre variables que originalmente tienen distintas escalas.

## Fórmula de la Normalización Min-Max.

- ▶ Dada una variable  $x$  con valores mínimos y máximos  $x_{\min}$  y  $x_{\max}$ , la normalización Min-Max se define mediante:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- ▶  $x'$ : valor normalizado en el rango  $[0, 1]$
- ▶  $x$ : valor original
- ▶  $x_{\min}$ ,  $x_{\max}$ : mínimo y máximo de la variable

## Normalización a un Rango Arbitrario.

- ▶ Si se desea normalizar a un rango  $[a, b]$ , la fórmula se adapta a:

$$x' = a + \frac{(x - x_{\min}) \cdot (b - a)}{x_{\max} - x_{\min}}$$

- ▶ Esto permite centrar los datos en rangos distintos, por ejemplo  $[-1, 1]$  o  $[0, 100]$ , según los requerimientos del problema.

## Ejemplo Numérico.

- Considere los valores originales:

$$\{50, 20, 30, 80, 100\}$$

$$x_{\text{mín}} = 20, \quad x_{\text{máx}} = 100$$

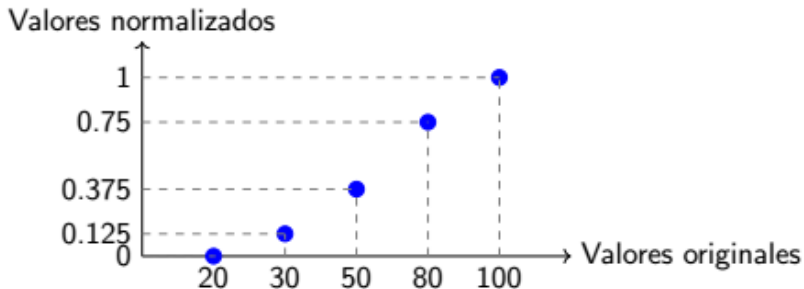
- Aplicando la normalización Min-Max:

$$50' = \frac{50 - 20}{100 - 20} = \frac{30}{80} = 0.375$$

- Los valores normalizados serían aproximadamente:

$$\{0.375, 0, 0.125, 0.75, 1\}$$

## Visualización Gráfica.



Se observa cómo los valores quedan reescalados al rango  $[0, 1]$ .



## Conjunto de Vectores de Características.

- ▶ En problemas de Machine Learning, la información suele representarse como una matriz de datos:

$$X \in \mathbb{R}^{M \times N}$$
$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1N} \\ x_{21} & x_{22} & \cdots & x_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ x_{M1} & x_{M2} & \cdots & x_{MN} \end{bmatrix}$$

- ▶  $M$ : **número de muestras u observaciones.**
- ▶  $N$ : número de características o atributos.
- ▶ Cada fila:

$$\mathbf{x}^{(i)} = [x_{i1}, x_{i2}, \dots, x_{iN}]$$

corresponde a un vector de características del ejemplo  $i$ .

- ▶ Para evitar que la escala de las características afecte el rendimiento de los modelos, **se normaliza cada característica individualmente.**
- ▶ Para cada característica  $j = 1, \dots, N$ :

$$x_{\min}^{(j)} = \min_{1 \leq i \leq M} x_{ij}$$

$$x_{\max}^{(j)} = \max_{1 \leq i \leq M} x_{ij}$$

- ▶ **Cada componente** se normaliza como:

$$x'_{ij} = \frac{x_{ij} - x_{\min}^{(j)}}{x_{\max}^{(j)} - x_{\min}^{(j)}}$$

## Ejemplo Numérico.

- Sea la matriz de datos:

$$X = \begin{bmatrix} 2 & 100 \\ 4 & 300 \\ 6 & 400 \end{bmatrix}$$

- **Primer característica (columna 1):**

$$x_{\min}^{(1)} = 2, \quad x_{\max}^{(1)} = 6$$

$$2' = \frac{2 - 2}{4} = 0$$

$$4' = \frac{4 - 2}{4} = 0.5$$

$$6' = \frac{6 - 2}{4} = 1$$

► Segunda característica (columna 2):

$$x_{\min}^{(2)} = 100, \quad x_{\max}^{(2)} = 400$$

$$100' = \frac{100 - 100}{300} = 0$$

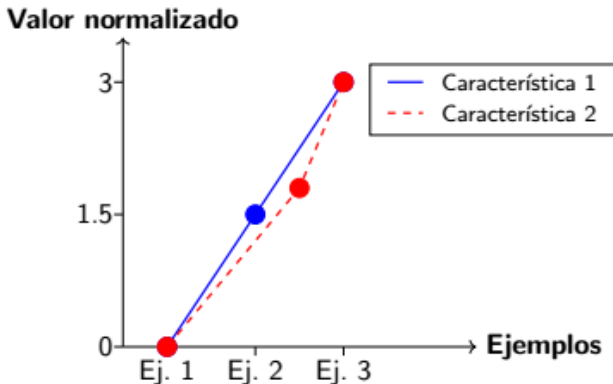
$$300' = \frac{300 - 100}{300} = 0.666$$

$$400' = \frac{400 - 100}{300} = 1$$

► Matriz normalizada resultante:

$$X' = \begin{bmatrix} 0 & 0 \\ 0.5 & 0.666 \\ 1 & 1 \end{bmatrix}$$

## Visualización Gráfica.



Cada característica se reescala de forma independiente.

## Uso en Entrenamiento y Predicción.

- ▶ Los valores  $x_{\min}^{(j)}$  y  $x_{\max}^{(j)}$  se calculan **usando sólo el conjunto de entrenamiento**.
- ▶ Durante la predicción, se usa la misma normalización:

$$x'_{*j} = \frac{x_{*j} - x_{\min}^{(j)}}{x_{\max}^{(j)} - x_{\min}^{(j)}}$$

para cualquier nuevo ejemplo  $x_*$ .

- ▶ De esta manera se asegura coherencia entre entrenamiento y producción.

## Ventajas y Desventajas.

### Ventajas:

- ▶ Sencilla de implementar.
- ▶ Conserva la forma de la distribución original.
- ▶ Útil para algoritmos sensibles a la escala.

### Desventajas:

- ▶ **Sensible a valores atípicos.**
- ▶ Si aparecen nuevos valores fuera del rango original, pueden quedar fuera del intervalo deseado.

La normalización Min-Max permite que todas las variables contribuyan de forma equilibrada, evitando dominancia por diferencias de escala.

## Normalización Z-score

- ▶ La técnica de normalización **Z-score transforma los datos para que tengan media cero y desviación estándar uno.**
- ▶ Esta técnica centra los datos y ajusta su dispersión, permitiendo que las características contribuyan de forma equilibrada independientemente de su escala original.



## Fórmula de la Normalización Z-score.

- ▶ Dada una variable  $x$  con media  $\mu$  y desviación estándar  $\sigma$ , la normalización Z-score se define mediante:

$$z = \frac{x - \mu}{\sigma}$$

- ▶  $z$ : valor normalizado.
- ▶  $x$ : valor original.
- ▶  $\mu$ : media de la variable.
- ▶  $\sigma$ : desviación estándar de la variable.

## Ejemplo Numérico.

- Considere los valores originales:

$$\{10, 12, 14, 16, 18\}$$

- **Media:**

$$\mu = \frac{10 + 12 + 14 + 16 + 18}{5} = 14$$

- **Desviación estándar:**

$$\begin{aligned}\sigma &= \sqrt{\frac{(10 - 14)^2 + (12 - 14)^2 + (14 - 14)^2 + (16 - 14)^2 + (18 - 14)^2}{5}} \\ &= \sqrt{\frac{16 + 4 + 0 + 4 + 16}{5}} = \sqrt{8} \approx 2.828\end{aligned}$$

## Cálculo de los valores normalizados.

$$10' = \frac{10 - 14}{2.828} \approx -1.414$$

$$12' = \frac{12 - 14}{2.828} \approx -0.707$$

$$14' = \frac{14 - 14}{2.828} = 0$$

$$16' = \frac{16 - 14}{2.828} \approx 0.707$$

$$18' = \frac{18 - 14}{2.828} \approx 1.414$$

Los datos quedan centrados en cero y con una dispersión estándar unitaria.

## Conjunto de Vectores de Características.

- ▶ En problemas de Machine Learning, la información se representa como una matriz:

$$X \in \mathbb{R}^{M \times N}$$
$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1N} \\ x_{21} & x_{22} & \cdots & x_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ x_{M1} & x_{M2} & \cdots & x_{MN} \end{bmatrix}$$

- ▶ Cada columna es una característica.
- ▶ **Cada fila es una muestra o vector de características.**

- ▶ Para cada característica  $j = 1, \dots, N$ :

$$\mu^{(j)} = \frac{1}{M} \sum_{i=1}^M x_{ij}$$

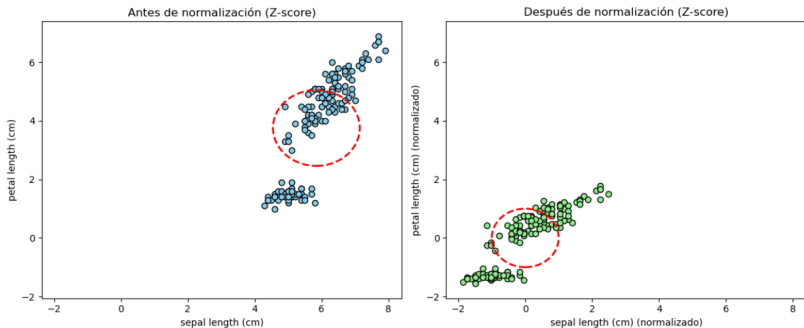
$$\sigma^{(j)} = \sqrt{\frac{1}{M} \sum_{i=1}^M (x_{ij} - \mu^{(j)})^2}$$

- ▶ Cada elemento se normaliza como:

$$z_{ij} = \frac{x_{ij} - \mu^{(j)}}{\sigma^{(j)}}$$

- ▶ De este modo, **cada característica tendrá media cero y desviación estándar uno.**

## Visualización Gráfica.



## Ventajas y Desventajas.

### Ventajas:

- ▶ Centra los datos en torno a cero.
- ▶ Hace que todas las características tengan la misma escala de dispersión.
- ▶ Útil para algoritmos basados en distancias y gradientes.

### Desventajas:

- ▶ **Sensible a valores atípicos**, que afectan la media y desviación estándar.

La normalización Z-score es especialmente útil cuando los datos siguen aproximadamente una distribución normal.

## Normalización por Norma (Vector Normalization)

- ▶ La normalización por norma **transforma cada vector de características para que tenga longitud (o norma) igual a 1.**
- ▶ Es útil cuando interesa comparar la dirección de los vectores, independientemente de su magnitud.



## Fórmula de la Normalización por Norma.

- ▶ Sea un vector de características:

$$\mathbf{x} = [x_1, x_2, \dots, x_N]$$

- ▶ Se calcula su norma (usualmente norma euclídea):

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \dots + x_N^2}$$

- ▶ Cada componente se normaliza como:

$$x'_i = \frac{x_i}{\|\mathbf{x}\|}, \quad \forall i = 1, \dots, N$$

## Ejemplo Numérico.

- ▶ Sea el vector:

$$\mathbf{x} = [3, 4]$$

- ▶ Norma euclídea:

$$\|\mathbf{x}\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

- ▶ Vector normalizado:

$$3' = \frac{3}{5} = 0.6$$

$$4' = \frac{4}{5} = 0.8$$

$$\mathbf{x}' = [0.6, 0.8]$$

## Conjunto de Vectores de Características.

- ▶ En un conjunto de datos representado por una matriz:

$$X \in \mathbb{R}^{M \times N}$$

cada fila corresponde a un vector de características  $\mathbf{x}^{(i)}$ .

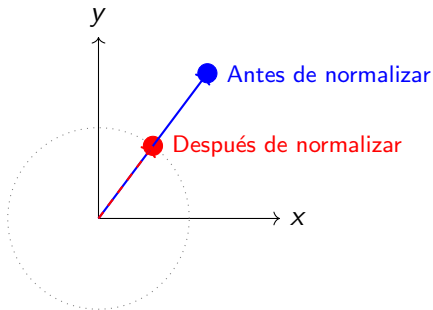
- ▶ **Se normaliza cada fila** por su norma:

$$\|\mathbf{x}^{(i)}\| = \sqrt{\sum_{j=1}^N (x_{ij})^2}$$

- ▶ Y luego:

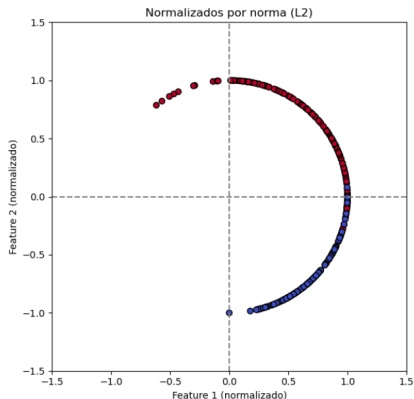
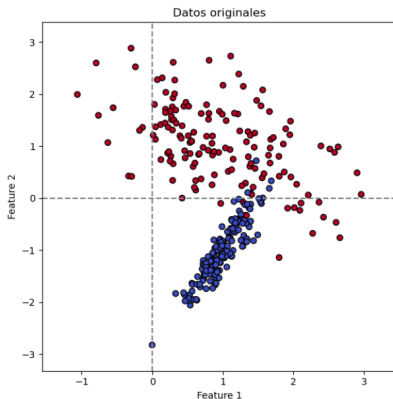
$$x'_{ij} = \frac{x_{ij}}{\|\mathbf{x}^{(i)}\|}$$

## Visualización Gráfica.



Se observa que el vector normalizado queda sobre el círculo de radio 1.

## Visualización Gráfica.



## Ventajas y Desventajas.

### Ventajas:

- ▶ Hace que todos los vectores tengan la misma magnitud.
- ▶ Resalta la dirección relativa de los datos.
- ▶ Útil para modelos basados en similitud de coseno.

### Desventajas:

- ▶ Puede perder información de magnitud importante.
- ▶ No es adecuada si el tamaño absoluto de los valores es relevante.

### Común en procesamiento de texto.

## Escalado Robusto (Robust Scaling)

- ▶ **Transforma los datos usando la mediana y el rango intercuartílico (IQR).**
- ▶ **Reduce la influencia de valores atípicos**, ya que no depende de la media ni de la desviación estándar.

## Fórmula del Escalado Robusto.

- ▶ Dada una variable  $x$ :

$$x' = \frac{x - \text{mediana}(x)}{\text{IQR}(x)}$$

donde:

$$\text{IQR}(x) = Q_3 - Q_1$$

- ▶  $Q_1$ : primer cuartil, es el valor que deja el 25 % de los datos por debajo.
- ▶  $Q_3$ : tercer cuartil es el valor que deja el 75 % de los datos por debajo.
- ▶ El resultado es un **centrado en la mediana y un reescalado según la dispersión central.**



## Ejemplo.

- Valores originales (dos características, cinco muestras):

|       | C1  | C2  |
|-------|-----|-----|
| Ej. 1 | 10  | 20  |
| Ej. 2 | 12  | 25  |
| Ej. 3 | 14  | 30  |
| Ej. 4 | 16  | 35  |
| Ej. 5 | 100 | 200 |

- **Mediana e IQR:**

C1:  $Q_1 = 12$ ,  $Q_3 = 16$ , Mediana = 14, IQR = 4

C2:  $Q_1 = 25$ ,  $Q_3 = 35$ , Mediana = 30, IQR = 10

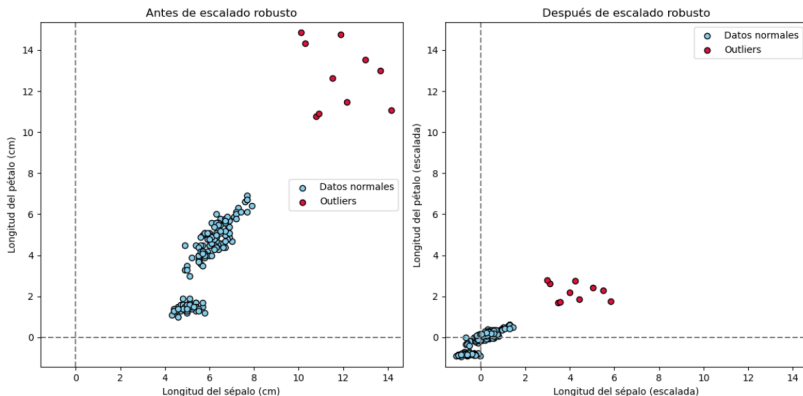
- Normalización robusta (mediana e IQR):

$$14' = \frac{14 - 14}{4} = 0, \quad 100' = \frac{100 - 14}{4} = 21.5$$

$$30' = \frac{30 - 30}{10} = 0, \quad 200' = \frac{200 - 30}{10} = 17$$

- Se observa que los valores atípicos quedan lejos, pero los valores centrales se centran en torno a 0.

## Visualización Gráfica.



## Ventajas y Desventajas.

### Ventajas:

- ▶ Menor sensibilidad a valores atípicos.
- ▶ Preserva la estructura central de los datos.

### Desventajas:

- ▶ No reescala necesariamente a un rango fijo.
- ▶ Puede ser menos intuitivo que Min-Max o Z-score.

## Escalado Decimal (Decimal Scaling)

- ▶ **Reescala los datos dividiéndolos por una potencia de 10**, de forma que todos queden en un rango determinado.
- ▶ Es sencillo y útil cuando los datos tienen magnitudes muy grandes.

## Fórmula del Escalado Decimal.

- ▶ Dada una variable  $x$ :

$$x' = \frac{x}{10^j}$$

donde:

$$j = \min\{j \in \mathbb{Z} : \max(|x'|) < 1\}$$

- ▶  $j$ : número de dígitos necesarios para asegurar que todos los valores normalizados sean menores que 1 en valor absoluto.

## Ejemplo.

- ▶ Valores originales (dos características):

|       | C1 | C2   |
|-------|----|------|
| Ej. 1 | 12 | 450  |
| Ej. 2 | 25 | 800  |
| Ej. 3 | 9  | 1200 |
| Ej. 4 | 17 | 600  |

- ▶ Cálculo de  $j$ :

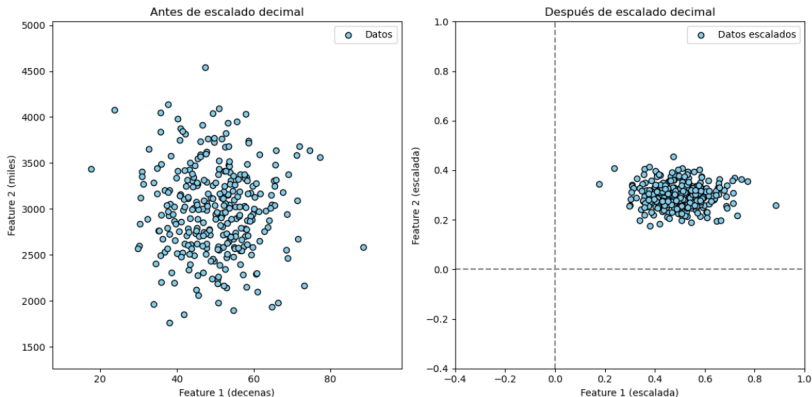
$$C1: \max = 25, j = 2 \quad \left( \frac{25}{100} = 0.25 < 1 \right)$$

$$C2: \max = 1200, j = 4 \quad \left( \frac{1200}{10000} = 0.12 < 1 \right)$$

- ▶ Valores normalizados:

$$25' = \frac{25}{100} = 0.25, \quad 1200' = \frac{1200}{10000} = 0.12$$

## Visualización Gráfica.



Los datos originales se reescalan para que queden menores que 1 en módulo.

## Ventajas y Desventajas.

### Ventajas:

- ▶ Extremadamente simple y rápido de calcular.
- ▶ Útil cuando las magnitudes tienen gran diferencia.

### Desventajas:

- ▶ **No considera la distribución estadística de los datos.**
- ▶ Puede no ser adecuado si los datos contienen valores atípicos.



## Transformación para Distribución

- ▶ **Modifica la forma de la distribución de los datos para aproximarla a una distribución deseada**, frecuentemente normal (gaussiana).
- ▶ Es útil para mejorar el rendimiento de algoritmos que suponen normalidad o simetría.

## Ejemplos Típicos de Transformaciones

- ▶ **Logarítmica:**

$$x' = \log(x)$$

- ▶ **Box-Cox (para  $x > 0$ ):**

$$x' = \begin{cases} \frac{x^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \log(x), & \lambda = 0 \end{cases}$$

- ▶ **Yeo-Johnson (para  $x \in \mathbb{R}$  – extiende Box-Cox):**

$$x' = \begin{cases} \frac{[(x+1)^\lambda - 1]}{\lambda}, & x \geq 0, \lambda \neq 0 \\ \log(x+1), & x \geq 0, \lambda = 0 \\ \frac{[-(-x+1)^{2-\lambda} + 1]}{2-\lambda}, & x < 0, \lambda \neq 2 \\ -\log(-x+1), & x < 0, \lambda = 2 \end{cases}$$

Estas transformaciones reducen asimetría y curtosis, acercando la distribución a una forma más simétrica.

## Ejemplo con datos de una variable.

- ▶ Valores originales (positivos y asimétricos):

$$\{1, 2, 5, 10, 50\}$$

- ▶ Transformación logarítmica:

$$1' = \log(1) = 0$$

$$2' = \log(2) \approx 0.69$$

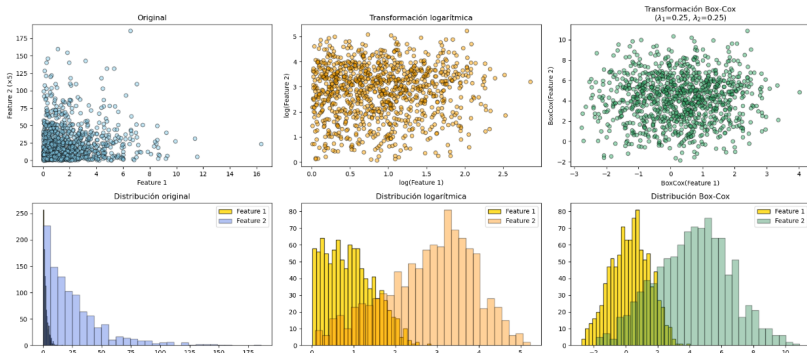
$$5' = \log(5) \approx 1.61$$

$$10' = \log(10) \approx 2.30$$

$$50' = \log(50) \approx 3.91$$

- ▶ Los valores grandes se comprimen más, reduciendo asimetría.

## Visualización Gráfica.



Se observa cómo la distribución original sesgada se vuelve más simétrica tras la transformación.

## Ventajas y Desventajas.

### Ventajas:

- ▶ Reduce asimetría y curtosis.
- ▶ Puede mejorar la eficacia de modelos sensibles a la normalidad.

### Desventajas:

- ▶ **Cambia la interpretación de las variables.**
- ▶ Algunas transformaciones requieren datos positivos.
- ▶ La selección del mejor método puede necesitar validación.

**Ver código python.**

## Técnicas de normalización interna.

- ▶ Las técnicas de normalización interna **buscan mejorar el entrenamiento de redes neuronales** al estabilizar y controlar la distribución de las activaciones intermedias.
- ▶ Su propósito principal es **reducir el desplazamiento covariante interno** (internal covariate shift), es decir, los **cambios en la distribución de las entradas a cada capa** a medida que los parámetros de las capas anteriores cambian durante el entrenamiento.
- ▶ Al mantener una distribución estable (normalizada) de activaciones, se facilita el aprendizaje, se aceleran la convergencia y se mejora la generalización.

## Batch Normalization.

- ▶ Batch Normalization (BN) es una técnica de normalización interna **aplicada durante el entrenamiento** de redes neuronales profundas.
- ▶ Su objetivo es estabilizar y acelerar el entrenamiento al normalizar las activaciones de cada capa, **usando estadísticas del minibatch actual**.
- ▶ Fue introducida por Ioffe y Szegedy en 2015.

- ▶ En el entrenamiento de **redes profundas**, la **distribución de las activaciones internas cambia constantemente**  $\Rightarrow$  *“desplazamiento covariante interno”*.
- ▶ Este fenómeno dificulta el aprendizaje, ya que las capas deben adaptarse continuamente a nuevas distribuciones de entrada.
- ▶ **Batch Normalization (BN)** mitiga este problema al **mantener las activaciones con media cero y varianza unitaria dentro de cada minibatch**.



**Proceso:** Para cada neurona y minibatch, BN realiza los siguientes pasos:.

1. **Cálculo de la media:**  $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$
2. **Cálculo de la varianza:**  $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$
3. **Normalización:**

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

4. **Reescalado y desplazamiento:**  $y_i = \gamma \hat{x}_i + \beta$ 
  - ▶  $\gamma, \beta$ : parámetros entrenables que restauran capacidad representativa.
  - ▶  $\epsilon$ : pequeño valor para estabilidad numérica.

**Ejemplo Numérico.** Considere una activación de **una neurona en un minibatch** de tamaño  $m = 4$ :

$$x = \{10, 12, 14, 16\}$$

► **Paso 1:** Media:  $\mu_B = \frac{10+12+14+16}{4} = 13$

► **Paso 2:** Varianza

$$\sigma_B^2 = \frac{(10-13)^2 + (12-13)^2 + (14-13)^2 + (16-13)^2}{4} = \frac{26}{4} = 6.5$$

► **Paso 3:** Normalización ( $\epsilon = 10^{-5}$ ):

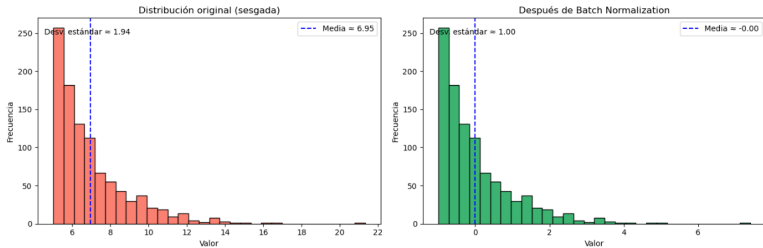
$$\hat{x}_1 = \frac{10 - 13}{\sqrt{6.5 + \epsilon}} \approx -1.17, \quad \hat{x}_4 \approx 1.17$$

► **Paso 4:** Reescalado con  $\gamma = 1.0$ ,  $\beta = 0.0, \Rightarrow y_i = \hat{x}_i$

Las activaciones quedan centradas en 0 y con varianza  $\simeq 1$ .

## Visualización Gráfica.

Efecto de Batch Normalization sobre una distribución sesgada



- ▶ Se muestra cómo una distribución sesgada y desbalanceada se transforma en una distribución normalizada centrada en cero y con varianza unitaria.

## Proceso paso a paso en una red neuronal.

Se aplica sobre la pre-activación de cada capa oculta, **para cada neurona  $j$** :

$$z_j^{[\ell]} = W_j^{[\ell]} a^{[\ell-1]} + b_j^{[\ell]}$$

**Para cada capa oculta  $\ell$ , y para cada neurona  $j$ , se siguen los pasos:**

1. Calcular la media del mini-batch:

$$\mu_{j,\text{batch}}^{[\ell]} = \frac{1}{m} \sum_{i=1}^m z_j^{[\ell](i)}$$

- ▶  $z_j^{[\ell](i)}$ : preactivación de la neurona  $j$  en la capa  $\ell$  para el ejemplo  $i$ -ésimo del mini-batch.
- ▶  $m$ : total de muestras usadas (tamaño del mini-batch).

2. Calcular la varianza del mini-batch:

$$\sigma_{j,\text{batch}}^{2[\ell]} = \frac{1}{m} \sum_{i=1}^m (z_j^{[\ell](i)} - \mu_{j,\text{batch}}^{[\ell]})^2$$

3. **Normalizar** la pre-activación de cada neurona  $j$ :

$$\hat{z}_j^{[\ell](i)} = \frac{z_j^{[\ell](i)} - \mu_{j,\text{batch}}^{[\ell]}}{\sqrt{\sigma_{j,\text{batch}}^{2[\ell]} + \varepsilon}}$$

4. Aplicar **escalado y desplazamiento** aprendibles (uno por neurona):

$$\tilde{z}_j^{[\ell](i)} = \gamma_j^{[\ell]} \cdot \hat{z}_j^{[\ell](i)} + \beta_j^{[\ell]}$$

5. **Calcular la activación:**

$$a_j^{[\ell](i)} = \phi(\tilde{z}_j^{[\ell](i)})$$

Este proceso se repite para cada neurona  $j$  de cada capa oculta  $\ell$ .

## Ejemplo en una red con dos capas ocultas.

### Capa oculta 1:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$z^{[1]} \xrightarrow{\text{Normalización}} \hat{z}^{[1]} \xrightarrow{\text{Escalado y Desplazamiento}} \tilde{z}^{[1]} = \gamma^{[1]} \cdot \hat{z}^{[1]} + \beta^{[1]}$$

$$a^{[1]} = \phi(\tilde{z}^{[1]})$$

### Capa oculta 2:

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$z^{[2]} \xrightarrow{\text{Normalización}} \hat{z}^{[2]} \xrightarrow{\text{Escalado y Desplazamiento}} \tilde{z}^{[2]} = \gamma^{[2]} \cdot \hat{z}^{[2]} + \beta^{[2]}$$

$$a^{[2]} = \phi(\tilde{z}^{[2]})$$

## Resumen general.

- ▶ En cada capa oculta, Batch Normalization transforma:

$$z^{[\ell]} \xrightarrow{\text{Normalización}} \hat{z}^{[\ell]} \xrightarrow{\text{Escalado y Desplazamiento}} \tilde{z}^{[\ell]} = \gamma^{[\ell]} \cdot \hat{z}^{[\ell]} + \beta^{[\ell]}$$

- ▶ Después aplica:

$$a^{[\ell]} = \phi(\tilde{z}^{[\ell]})$$

- ▶ Donde:
  - ▶  $\gamma^{[\ell]}$  y  $\beta^{[\ell]}$ : parámetros aprendibles.
  - ▶  $\varepsilon$ : pequeño valor positivo para evitar divisiones por cero.

## Batch Normalization: Durante la Inferencia.

- ▶ Durante el entrenamiento:
  - ▶ Se usa la media  $\mu_B$  y varianza  $\sigma_B^2$  del minibatch.
- ▶ Durante la inferencia (test):
  - ▶ Se usan **estadísticas acumuladas globales** (media y varianza promedio durante el entrenamiento).
  - ▶ Esto asegura comportamiento consistente y determinista.



## Beneficios de Batch Normalization.

- ▶ Reduce el desplazamiento covariante interno.
- ▶ Acelera el entrenamiento.
- ▶ Permite usar tasas de aprendizaje mayores.
- ▶ Actúa como regularizador adicional → reduce necesidad de dropout.
- ▶ Reduce sensibilidad a la inicialización de parámetros.
- ▶ Mejora la generalización en muchos casos.

BatchNorm es clave en redes profundas modernas para obtener entrenamiento más rápido y estable.

## Batch Normalization: Limitaciones y Variantes.

### Limitaciones:

- ▶ Depende del tamaño del minibatch (funciona peor con minibatches pequeños).
- ▶ No siempre es efectiva para secuencias o entradas con longitud variable.

### Variantes comunes:

- ▶ **Layer Normalization:** normaliza sobre las activaciones de una capa completa (útil para NLP).
- ▶ **Instance Normalization:** útil para visión por computadora.
- ▶ **Group Normalization:** alternativa cuando el batch size es pequeño.

## Layer Normalization.

- ▶ Layer Normalization (LN) es una técnica de normalización interna aplicada en redes neuronales profundas.
- ▶ Su objetivo es estabilizar y acelerar el entrenamiento al normalizar las activaciones de cada capa por cada ejemplo individual, en vez de usar estadísticas del minibatch.
- ▶ Fue introducida por Ba, Kiros y Hinton en 2016.

- ▶ Layer Normalization aborda el *desplazamiento covariante interno*, igual que BatchNorm.
- ▶ Sin embargo, LN normaliza cada muestra por separado, **usando la media y varianza calculadas sobre las neuronas de una capa.**
- ▶ Esto la hace especialmente útil en redes recurrentes (RNNs) y modelos con secuencias, donde los tamaños de minibatch pueden ser pequeños o variables.

**Proceso:** Para cada muestra y capa, Layer Normalization realiza los siguientes pasos:

Suponga que una capa tiene  $H$  neuronas y, para una muestra individual (no todas las muestras del minibatch), las activaciones son:

$$\mathbf{z} = [z_1, z_2, \dots, z_H]$$

1. **Cálculo de la media:**  $\mu_L = \frac{1}{H} \sum_{j=1}^H z_j$
2. **Cálculo de la varianza:**  $\sigma_L^2 = \frac{1}{H} \sum_{j=1}^H (z_j - \mu_L)^2$
3. **Normalización:**  $\hat{z}_j = \frac{z_j - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}}$
4. **Reescalado y desplazamiento:**  $\tilde{z}_j = \gamma_j \cdot \hat{z}_j + \beta_j$

$\gamma_j, \beta_j$ : parámetros aprendibles por cada neurona.

**Ejemplo Numérico:** Suponga una capa con  $H = 4$  neuronas, y activaciones de una muestra:

$$\mathbf{z} = [5, 7, 9, 11]$$

► **Paso 1:** Media:  $\mu_L = \frac{5+7+9+11}{4} = 8$

► **Paso 2:** Varianza:

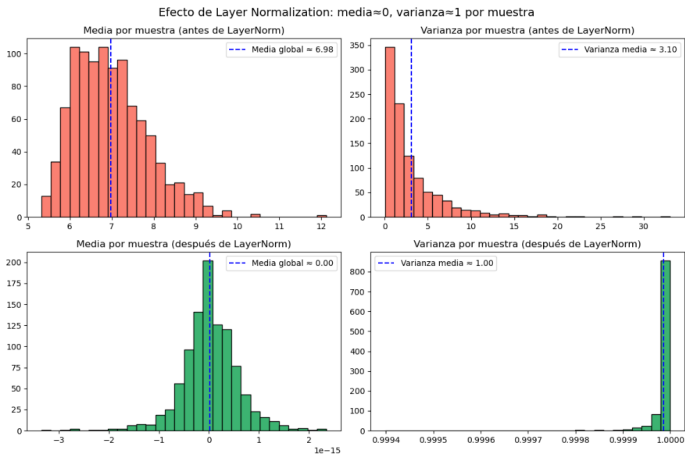
$$\sigma_L^2 = \frac{(5-8)^2 + (7-8)^2 + (9-8)^2 + (11-8)^2}{4} = \frac{20}{4} = 5$$

► **Paso 3:** Normalización ( $\varepsilon = 10^{-5}$ ):

$$\hat{z}_1 = \frac{5-8}{\sqrt{5+10^{-5}}} \approx -1.34, \quad \hat{z}_4 \approx 1.34$$

► **Paso 4:** Reescalado con  $\gamma = 1$ ,  $\beta = 0$ :  $\tilde{z}_j = \hat{z}_j$

## Visualización Gráfica.



Se observa cómo las activaciones del mismo ejemplo se centran en cero y adquieren varianza unitaria.

## Proceso paso a paso en una red neuronal.

En cada muestra del batch y capa:

$$z^{[\ell]} = W^{[\ell]} a^{[\ell-1]} + b^{[\ell]}$$

1. Calcular media de las activaciones de todas las neuronas de la capa:

$$\mu^{[\ell]} = \frac{1}{H} \sum_{j=1}^H z_j^{[\ell]}$$

2. Calcular varianza:

$$\sigma^{2[\ell]} = \frac{1}{H} \sum_{j=1}^H (z_j^{[\ell]} - \mu^{[\ell]})^2$$



## 3. Normalizar:

$$\hat{z}_j^{[\ell]} = \frac{z_j^{[\ell]} - \mu^{[\ell]}}{\sqrt{\sigma^{2[\ell]} + \varepsilon}}$$

## 4. Escalado y desplazamiento:

$$\tilde{z}_j^{[\ell]} = \gamma_j^{[\ell]} \cdot \hat{z}_j^{[\ell]} + \beta_j^{[\ell]}$$

## 5. Activación:

$$a_j^{[\ell]} = \phi(\tilde{z}_j^{[\ell]})$$

## Resumen general de Layer Normalization.

- ▶ En cada capa y muestra:

$$z^{[\ell]} \rightarrow \hat{z}^{[\ell]} \rightarrow \tilde{z}^{[\ell]} = \gamma^{[\ell]} \cdot \hat{z}^{[\ell]} + \beta^{[\ell]}$$

y después:

$$a^{[\ell]} = \phi(\tilde{z}^{[\ell]})$$

- ▶  $\gamma^{[\ell]}, \beta^{[\ell]}$ : aprendibles (uno por neurona).
- ▶  $\varepsilon$ : evita divisiones por cero.

## Ventajas y Observaciones de Layer Normalization.

### Ventajas:

- ▶ No depende del tamaño del minibatch.
- ▶ Funciona bien en secuencias y redes recurrentes.
- ▶ Reduce desplazamiento covariante interno.

### Observaciones:

- ▶ Menos efectiva que BatchNorm en algunas redes convolucionales.
- ▶ Usa estadísticas de la muestra individual, no del minibatch.

**Ver código python.**

## Cálculo de las estadísticas. BatchNorm vs LayerNorm

### ► Batch Normalization (BatchNorm):

- Calcula la media y varianza para cada neurona de la capa.
- Usa todas las muestras del mini-batch:

$$\mu_{j,\text{batch}}^{[\ell]} = \frac{1}{m} \sum_{i=1}^m z_j^{[\ell](i)}$$

- Depende del tamaño del batch.

### ► Layer Normalization (LayerNorm):

- Calcula la media y varianza sobre todas las neuronas de la capa, pero sólo para una muestra  $i$ :

$$\mu^{[\ell](i)} = \frac{1}{n^{[\ell]}} \sum_{j=1}^{n^{[\ell]}} z_j^{[\ell](i)}$$

- Se hace de forma independiente para cada muestra.
- **No depende del batch**; funciona incluso con batch size = 1.

- $m$  es el tamaño del mini-batch y
- $n^{[\ell]}$  es el número de neuronas en la capa  $\ell$ .

## Instance Normalization.

- ▶ Instance Normalization (IN) es una técnica de normalización interna usada principalmente en tareas de visión por computadora y estilo de imagen.
- ▶ **Normaliza cada muestra y cada canal de forma independiente**, usando estadísticas calculadas solo sobre el espacio espacial (alto  $\times$  ancho).
- ▶ Fue propuesta por Ulyanov et al. en 2016, y es clave para transferencias de estilo y generación de imágenes.

- ▶ Instance Normalization aborda el problema del *desplazamiento covariante interno*, igual que BN y LN.
- ▶ Sin embargo, IN normaliza cada canal de cada ejemplo de forma independiente, sin usar estadísticas del minibatch ni de toda la capa.
- ▶ Es útil cuando se quiere eliminar dependencias de contraste o iluminación en cada imagen.

## Proceso.

- ▶ Sea un tensor de activaciones de una capa convolucional:

$$x \in \mathbb{R}^{N \times C \times H \times W}$$

- ▶ Donde:
  - ▶  $n$ : índice del ejemplo dentro del mini-batch (por ejemplo, la imagen número  $n$ ).
  - ▶  $c$ : índice del canal (por ejemplo, R, G, B si tienes 3 canales; o un canal intermedio en la red).
  - ▶  $h$ : índice de la altura espacial de la imagen (número de filas).
  - ▶  $w$ : índice del ancho espacial de la imagen (número de columnas).

# Normalización de Datos.

Para cada muestra  $n$  y canal  $c$ , IN realiza:

1. Calcula la media espacial:

$$\mu_{nc} = \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W x_{nchw}$$

2. Calcula la varianza espacial:

$$\sigma_{nc}^2 = \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W (x_{nchw} - \mu_{nc})^2$$

3. Normaliza:

$$\hat{x}_{nchw} = \frac{x_{nchw} - \mu_{nc}}{\sqrt{\sigma_{nc}^2 + \varepsilon}}$$

4. Reescala y desplaza:

$$y_{nchw} = \gamma_c \cdot \hat{x}_{nchw} + \beta_c$$

$\gamma_c, \beta_c$ : parámetros aprendibles por canal.

5. El resultado  $y_{nchw}$  se pasa a la siguiente capa de la red.



## Ejemplo Numérico (simplificado):

Suponga:

$$x_{1,1,:} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$$

(donde  $N = 1$ ,  $C = 1$ ,  $H = W = 2$ ).

► **Paso 1:** Media:

$$\mu_{11} = \frac{2 + 4 + 6 + 8}{4} = 5$$

► **Paso 2:** Varianza:

$$\sigma_{11}^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = \frac{9 + 1 + 1 + 9}{4} = 5$$

► **Paso 3:** Normalización ( $\varepsilon = 10^{-5}$ ):

$$\hat{x}_{1,1,1,1} = \frac{2-5}{\sqrt{5}} \approx -1.34$$

► **Paso 4:** Reescalado ( $\gamma = 1$ ,  $\beta = 0$ ):

$$y_{1,1,1,1} = -1.34$$

## Resumen general de Instance Normalization.

- ▶ Para cada muestra y canal:

$$x_{nchw} \xrightarrow{\text{Normalización}} \hat{x}_{nchw} \xrightarrow{\text{Escalado y Desplazamiento}} y_{nchw} = \gamma_c \cdot \hat{x}_{nchw} + \beta_c$$

- ▶  $\gamma_c, \beta_c$ : aprendibles (uno por canal).
- ▶ Se normaliza usando media y varianza calculadas solo sobre el canal de cada muestra y aplicando el escalado y desplazamiento ( $\gamma_c, \beta_c$ ).
- ▶ El resultado  $y_{nchw}$  se pasa a la siguiente capa de la red.

## Ventajas y Observaciones de Instance Normalization.

### Ventajas:

- ▶ **Independiente del tamaño de minibatch.**
- ▶ Muy útil en transferencia de estilo y generación de imágenes.
- ▶ Elimina diferencias de brillo y contraste entre imágenes.

### Limitaciones:

- ▶ **Puede eliminar información de contraste útil** para tareas de clasificación.
- ▶ Menos efectiva que BN o LN en redes puramente discriminativas.

## Group Normalization.

- ▶ Group Normalization (GN) es una técnica de normalización interna propuesta por Wu y He en 2018.
- ▶ Divide los canales de cada muestra en varios grupos y normaliza cada grupo de forma independiente.
- ▶ Fue diseñada para superar la dependencia del tamaño del minibatch, manteniendo ventajas de Batch y Layer Normalization.

- ▶ Group Normalization aborda el problema del *desplazamiento covariante interno*.
- ▶ A diferencia de BN, GN no utiliza estadísticas de todo el minibatch.
- ▶ Divide los canales  $C$  en  $G$  grupos de  $C/G$  canales cada uno, y normaliza cada grupo usando la media y varianza calculadas sobre las posiciones espaciales y los canales del grupo.

## Proceso.

- ▶ Sea un tensor de activaciones:

$$x \in \mathbb{R}^{N \times C \times H \times W}$$

- ▶ Donde:
  - ▶  $N$ : número de muestras (minibatch).
  - ▶  $C$ : número de canales.
  - ▶  $H, W$ : dimensiones espaciales.
- ▶ Se eligen  $G$  grupos.
- ▶ Cada grupo tiene  $C/G$  canales.

Para cada muestra  $n$  y cada grupo  $g$ , GN realiza:

1. Calcular la media:

$$\mu_{ng} = \frac{1}{m} \sum_{i=1}^{m=(C/G) \cdot H \cdot W} x_i$$

donde  $m$  es número de elementos en el grupo.

2. Calcular la varianza:  $\sigma_{ng}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{ng})^2$

3. Normalizar:

$$\hat{x}_i = \frac{x_i - \mu_{ng}}{\sqrt{\sigma_{ng}^2 + \varepsilon}}$$

4. Reescalar y desplazar:  $y_i = \gamma_c \cdot \hat{x}_i + \beta_c$   $\gamma_c, \beta_c$ : aprendibles (uno por canal).
5. El resultado se pasa a la siguiente capa de la red.

## Ejemplo Numérico (muy simplificado):

- ▶ Sponga:

$$N = 1, C = 4, H = W = 1, G = 2$$

- ▶ Tensor:

$$x_{1,::,:} = [2, 4, 6, 8]$$

- ▶ **Paso 1:** Dividir en grupos:

- ▶ Grupo 1: canales 1 y 2  $\rightarrow \{2, 4\}$
- ▶ Grupo 2: canales 3 y 4  $\rightarrow \{6, 8\}$

- ▶ **Grupo 1:**  $\mu = 3, \sigma^2 = 1$ .

$$\hat{x}_1 = \frac{2 - 3}{\sqrt{1}} = -1, \quad \hat{x}_2 = 1$$

- ▶ **Grupo 2:**  $\mu = 7, \sigma^2 = 1$ .

$$\hat{x}_3 = \frac{6 - 7}{1} = -1, \quad \hat{x}_4 = 1$$



## Resumen general de Group Normalization.

- ▶ Para cada muestra:
  - ▶ Dividir los  $C$  canales en  $G$  grupos.
  - ▶ Para cada grupo:

$$x_i \xrightarrow{\text{Normalización}} \hat{x}_i \xrightarrow{\text{Escalado y Desplazamiento}} y_i = \gamma_c \cdot \hat{x}_i + \beta_c$$

usando media y varianza calculadas sobre los canales del grupo y sus posiciones espaciales.

- ▶  $\gamma_c, \beta_c$ : aprendibles, uno por canal.

## Ventajas y Observaciones de Group Normalization.

### Ventajas:

- ▶ **Independiente del tamaño del minibatch.**
- ▶ Funciona bien en visión por computadora, especialmente con minibatches pequeños.
- ▶ Generaliza BN (si  $G = 1$  se parece a LN; si  $G = C$ , a IN).

### Limitaciones:

- ▶ Menos eficaz que BN cuando se dispone de minibatches grandes.
- ▶ **Requiere elegir un buen número de grupos  $G$ .**

## Técnicas de Entrenamiento en Machine Learning.

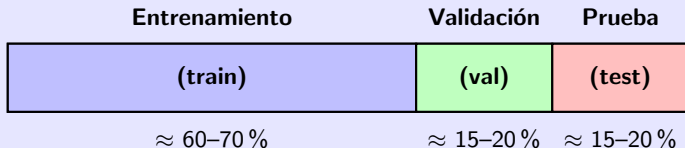
- ▶ Son métodos para organizar, dividir y utilizar los datos durante el entrenamiento de los modelos.
- ▶ Su objetivo principal es:
  - ▶ Mejorar la capacidad de generalización del modelo.
  - ▶ Evitar sobreajuste (overfitting).
  - ▶ Obtener una estimación más fiable del rendimiento.

## Técnicas más comunes:

- ▶ Hold-out (partición simple en train/validation/test).
- ▶ Validación cruzada ( $K$ -fold).
- ▶ Leave-One-Out Cross-Validation (LOOCV).
- ▶ Muestreo estratificado.
- ▶ Hold-out repetido.
- ▶ Bootstrap.
- ▶ Early stopping (detención temprana).

## Técnica Hold-out

- ▶ Es una técnica clásica de **partición fija** para dividir el conjunto de datos.
- ▶ Consiste en separar los datos una sola vez antes del entrenamiento en:



- ▶ Cada subconjunto tiene un propósito diferente para construir modelos que **generalicen bien**:
  - ▶ **Entrenamiento**: ajustar parámetros.
  - ▶ **Validación**: monitorear y ajustar hiperparámetros.
  - ▶ **Prueba**: evaluación final.

## 1. Conjunto de entrenamiento (train):

- ▶ Se utiliza para **ajustar los parámetros del modelo**.
- ▶ El modelo ve estos datos muchas veces.
- ▶ Se minimiza la función de pérdida:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$$

## 2. Conjunto de validación (val):

- ▶ Evalúa el rendimiento durante el entrenamiento.
- ▶ Ayuda a ajustar hiperparámetros y detectar sobreajuste.
- ▶ Se calculan métricas como:

$$\text{loss}_{val}, \quad \text{accuracy}_{val}$$

## 3. Conjunto de prueba (test):

- ▶ Se usa solo al final, nunca en el entrenamiento.
- ▶ Mide la capacidad real de generalización.

## Validación durante el entrenamiento.

- ▶ Durante el entrenamiento con la técnica Hold-out:
  1. Se entrena con el conjunto de entrenamiento.
  2. Tras cada epoch o cada cierto número de iteraciones, se evalúa el modelo en el conjunto de validación:

$$\text{loss}_{val}, \quad \text{accuracy}_{val}$$

3. Si la validación no mejora tras varias épocas, se puede aplicar **early stopping**.

### Ejemplo:

- ▶ Se ajusta el modelo usando  $\{x_{train}, y_{train}\}$ .
- ▶ Cada epoch, se evalúa sobre  $\{x_{val}, y_{val}\}$ .
- ▶ Si la pérdida de validación empieza a subir mientras la de entrenamiento sigue bajando  $\rightarrow$  posible sobreajuste.

## Pipeline de entrenamiento con Hold-out.

- ▶ En la técnica Hold-out, el pipeline básico es:

Datos totales  $\rightarrow$  Entrenamiento + Validación + Prueba  
ajustar parámetros      ajuste de hiperparámetros      evaluación final

- ▶ Así se garantiza que:
  - ▶ El modelo aprende patrones generales y no memoriza.
  - ▶ Se pueden **optimizar hiperparámetros sin tocar los datos de prueba.**
  - ▶ La métrica final refleja el rendimiento real en datos nunca vistos.

Separar correctamente los datos es esencial para evitar el sobreajuste y evaluar de forma fiable.



## Ventajas y Limitaciones de la técnica Hold-out

### Ventajas:

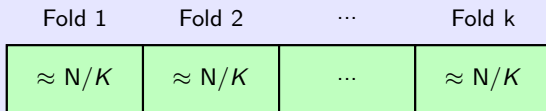
- ▶ Simplicidad: fácil de implementar y entender.
- ▶ Rápida: solo requiere entrenar el modelo una vez.
- ▶ Adecuada para conjuntos de datos muy grandes, donde repetir el entrenamiento sería costoso.

### Limitaciones:

- ▶ La evaluación depende de una única división del dataset → puede introducir **alta varianza**.
- ▶ Si el conjunto es pequeño, la estimación del error de generalización puede ser poco fiable.
- ▶ El resultado puede variar según cómo se dividan los datos.

## Técnica de Validación Cruzada

- ▶ La validación cruzada (CV) es una técnica para evaluar modelos de forma más robusta.
- ▶ Se basa en dividir el conjunto de datos ( $N$ ) varias veces en  $K$  partes llamadas **folds**.



- ▶ En cada iteración:
  - ▶ **Se usan algunos folds para entrenar.**
  - ▶ **Se reserva uno para validar.**
- ▶ Este proceso **se repite hasta que cada fold ha sido usado como validación una vez.**
- ▶ Al final, se promedia el rendimiento de todas las iteraciones para estimar la capacidad de generalización.

## División en $K$ folds

- ▶ El conjunto de datos se divide en  $K$  partes de tamaño similar.
- ▶ En cada iteración:

Entrenamiento:  $K - 1$  folds

Validación: 1 fold restante

- ▶ El modelo se entrena  $K$  veces, cambiando el fold de validación.

### Ejemplo con $K = 5$ :

- ▶ Iteración 1: folds 2,3,4,5 para entrenar; fold 1 para validar.
- ▶ Iteración 2: folds 1,3,4,5 para entrenar; fold 2 para validar.
- ▶ ...
- ▶ Iteración 5: folds 1,2,3,4 para entrenar; fold 5 para validar.

## Cálculo del rendimiento en Cross-Validation

- ▶ Tras entrenar  $K$  modelos, se calculan métricas como:

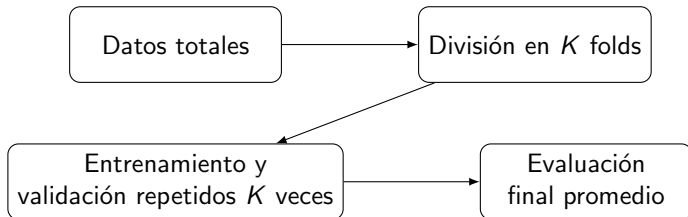
$$\text{loss}^{(1)}, \dots, \text{loss}^{(K)}; \quad \text{accuracy}^{(1)}, \dots, \text{accuracy}^{(K)}$$

- ▶ Se obtiene una estimación final promediando:

$$\overline{\text{loss}} = \frac{1}{K} \sum_{i=1}^K \text{loss}^{(i)} \quad \overline{\text{accuracy}} = \frac{1}{K} \sum_{i=1}^K \text{accuracy}^{(i)}$$

- ▶ Esto permite medir de forma más fiable la capacidad del modelo para generalizar.

## Pipeline con Validación Cruzada



- ▶ Ventajas:
  - ▶ **Reduce la dependencia de una sola división.**
  - ▶ Permite una mejor estimación del error de generalización.
- ▶ Después de la validación cruzada, se puede entrenar el modelo final usando todos los datos.

## Ventajas y Limitaciones de Validación Cruzada

### Ventajas:

- ▶ Más fiable que una única división (Hold-out).
- ▶ Reduce la varianza de la estimación del error.
- ▶ **Ideal cuando se dispone de pocos datos.**

### Limitaciones:

- ▶ **Mayor costo computacional:** hay que entrenar  $K$  modelos.
- ▶ En datasets muy grandes puede ser impráctico.

## Técnica Leave-One-Out Cross-Validation (LOOCV)

- ▶ Es un caso especial de validación cruzada.
- ▶ Consiste en hacer tantas particiones como instancias haya en el dataset:
  - ▶ En cada iteración, se deja **una única muestra** para validación.
  - ▶ El resto se usa para entrenamiento.
- ▶ Se repite este proceso  $N$  veces (siendo  $N$  el número total de datos).

Entrenamiento

Validación (1 muestra)



## Proceso de LOOCV

1. Se repite el entrenamiento  $N$  veces:
  - ▶ En cada iteración  $i$ , se deja fuera la muestra  $i$  para validación.
  - ▶ El modelo se entrena con el resto de  $N - 1$  muestras.
2. Tras cada iteración, se calcula la métrica de error o exactitud sobre la muestra de validación:

$$\text{error}^{(i)} = L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$$

3. Finalmente, se promedian los errores de todas las iteraciones:

$$\text{Error medio} = \frac{1}{N} \sum_{i=1}^N \text{error}^{(i)}$$



## Pipeline de entrenamiento con LOOCV

- ▶ El pipeline general:

Datos totales  $\rightarrow$   $\underbrace{\text{Iteración 1}}_{\substack{\text{1 muestra} \\ \text{de validación}}} + \underbrace{\text{Iteración 2}}_{\substack{\text{otra muestra} \\ \text{de validación}}} + \dots + \underbrace{\text{Iteración N}}_{\substack{\text{última muestra} \\ \text{de validación}}}$

- ▶ Tras entrenar y validar  $N$  veces, se calcula el error medio como métrica final.
- ▶ Garantiza que **cada muestra** sea usada exactamente una vez como validación.

LOOCV aprovecha al máximo los datos, especialmente útil para conjuntos pequeños.

## Ejemplo práctico de LOOCV

- Supóngase que se tiene un dataset con  $N = 5$  muestras:  $\{x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}, x^{(5)}\}$

1. Iteración 1: se entrena con  $\{2, 3, 4, 5\}$ ; se valida en 1.
2. Iteración 2: se entrena con  $\{1, 3, 4, 5\}$ ; se valida en 2.
3. Iteración 3: se entrena con  $\{1, 2, 4, 5\}$ ; se valida en 3.
4. Iteración 4: se entrena con  $\{1, 2, 3, 5\}$ ; se valida en 4.
5. Iteración 5: se entrena con  $\{1, 2, 3, 4\}$ ; se valida en 5.

- Finalmente:

$$\text{Error medio} = \frac{1}{5} \sum_{i=1}^5 \text{error}^{(i)}$$

## Ventajas y limitaciones de LOOCV

### Ventajas:

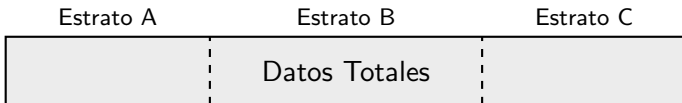
- ▶ Usa casi todos los datos para entrenar en cada iteración.
- ▶ **Muy útil en conjuntos de datos pequeños.**
- ▶ Baja varianza en la estimación del error.

### Limitaciones:

- ▶ **Costoso computacionalmente:** requiere entrenar el modelo  $N$  veces.
- ▶ En datasets grandes, puede ser impracticable.
- ▶ Puede tener alta varianza en modelos complejos o datos con ruido.

## Técnica Stratified Sampling (Muestreo Estratificado)

- ▶ Consiste en dividir el conjunto de datos en **estratos** o grupos según una característica clave (por ejemplo, la clase en un problema de clasificación).
- ▶ En cada estrato, se mantiene la misma proporción de clases en los subconjuntos de entrenamiento, validación y prueba.
- ▶ Garantiza que la distribución de las clases se mantenga similar a la original.



## Proceso de Stratified Sampling

1. Se identifica una variable clave para estratificar (por ejemplo, la clase).
  2. Se divide el dataset en estratos basados en esa variable.
  3. Se realiza la división en entrenamiento, validación y prueba dentro de cada estrato.
  4. Se combinan los subconjuntos estratificados de todos los estratos.
- ▶ De esta forma, **se preserva la proporción de clases en cada subconjunto.**
  - ▶ Especialmente **útil para datasets desbalanceados.**

## Pipeline con Stratified Sampling

- ▶ Pipeline típico:

Datos totales  $\xrightarrow{\text{estratificación}}$  Entrenamiento + Validación + Prueba  
manteniendo manteniendo manteniendo  
proporción proporción proporción

- ▶ Permite que todas las etapas del entrenamiento tengan datos representativos de todas las clases.
- ▶ Reduce el riesgo de que un subconjunto quede dominado por una clase.

## Ejemplo.

### Escenario inicial

- ▶ Dataset con 100 muestras:
  - ▶ Clase A: 70 muestras (70 %)
  - ▶ Clase B: 30 muestras (30 %)

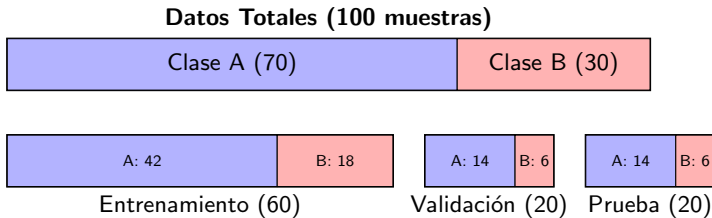
Se quiere dividir en:

- ▶ **Entrenamiento:** 60 % (60 muestras)
- ▶ **Validación:** 20 % (20 muestras)
- ▶ **Prueba:** 20 % (20 muestras)

Con **stratified sampling**, se mantienen las proporciones originales (70 % / 30 %) en cada subconjunto:

- ▶ Entrenamiento (60): 42 de clase A, 18 de clase B
- ▶ Validación (20): 14 de clase A, 6 de clase B
- ▶ Prueba (20): 14 de clase A, 6 de clase B

## Visualización del ejemplo.



Stratified Sampling mantiene las proporciones (70 % / 30 %) en todos los subconjuntos.



## Ventajas y limitaciones de Stratified Sampling

### Ventajas:

- ▶ Mantiene la distribución original de clases.
- ▶ **Mejora la estabilidad de las métricas** de evaluación.
- ▶ Es muy útil para datasets desbalanceados.

### Limitaciones:

- ▶ **Requiere conocer de antemano la variable para estratificar.**
- ▶ Menos efectivo si los datos no están claramente divididos en estratos.
- ▶ Más complejo de implementar que una división aleatoria simple.

## Técnica Repeated Hold-out

- ▶ Variante del hold-out clásico.
- ▶ Consiste en repetir varias veces el proceso de:
  1. Dividir aleatoriamente el dataset en:
    - ▶ Entrenamiento
    - ▶ Validación
    - ▶ Prueba
  2. Entrenar y evaluar el modelo.
- ▶ Al final, se calcula la media de las métricas (p.ej. accuracy, loss) obtenidas en todas las repeticiones.

*Reduce la varianza y proporciona una estimación más robusta del rendimiento del modelo.*

## Funcionamiento.

1. Se elige un número de repeticiones  $R$  (por ejemplo,  $R = 5$ ).
2. En cada repetición  $r = 1, \dots, R$ :
  - ▶ Se hace una partición aleatoria de los datos:

$$\text{Datos} \rightarrow \text{Train}_r + \text{Val}_r + \text{Test}_r$$

- ▶ Se entrena el modelo sobre  $\text{Train}_r$ .
  - ▶ Se evalúa sobre  $\text{Val}_r$  y  $\text{Test}_r$ .
3. Se obtienen métricas para cada repetición:

$$\text{accuracy}_r, \quad \text{loss}_r$$

4. Se calculan estadísticas resumidas:

$$\overline{\text{accuracy}} = \frac{1}{R} \sum_{r=1}^R \text{accuracy}_r$$

## Ventajas y limitaciones de Repeated Hold-out.

### Ventajas:

- ▶ **Reduce la dependencia de una sola partición aleatoria.**
- ▶ Estimación más robusta que el hold-out simple.
- ▶ Sencillo de implementar.

### Limitaciones:

- ▶ Requiere entrenar el modelo varias veces  $\Rightarrow$  mayor coste computacional.
- ▶ Aún puede presentar varianza, aunque menor.
- ▶ Las particiones pueden solaparse; algunas muestras pueden aparecer varias veces en test.

## Técnica Bootstrap

- ▶ Técnica de remuestreo que crea múltiples **muestras bootstrap** tomando aleatoriamente **con reemplazo**<sup>2</sup> del dataset original.
- ▶ Cada muestra tiene el mismo tamaño que el conjunto original, pero incluye repeticiones.
- ▶ Permite entrenar varios modelos y estimar métricas de forma más robusta.

*Muy usada para estimar la varianza y la estabilidad de los modelos.*

---

<sup>2</sup>Es decir, que cuando se selecciona un elemento, se vuelve a colocar en el conjunto antes de hacer la siguiente selección.

## Funcionamiento.

1. Se elige un número de muestras bootstrap  $B$  (p.ej.,  $B = 100$ ).
2. Para cada muestra  $b = 1, \dots, B$ :
  - ▶ Se genera una muestra aleatoria  $\text{Train}_b$  tomando  $N$  datos del dataset original, con reemplazo.
  - ▶ Los datos no elegidos pueden formar el conjunto de prueba  $\text{Test}_b$  (out-of-bag).
  - ▶ Se entrena el modelo sobre  $\text{Train}_b$  y se evalúa sobre  $\text{Test}_b$ .
3. Se obtienen métricas:

$$\text{accuracy}_b, \quad \text{loss}_b$$

4. Se calculan promedios:

$$\overline{\text{accuracy}} = \frac{1}{B} \sum_{b=1}^B \text{accuracy}_b$$

## Ejemplo de Bootstrap.

- ▶ Dataset original con  $N = 50$  muestras.
- ▶ Se generan  $B = 30$  conjuntos *bootstrap*.
- ▶ Cada conjunto bootstrap se obtiene tomando 50 muestras **con reemplazo** del dataset original.
- ▶ Para cada conjunto:
  - ▶ Se entrena un modelo.
  - ▶ Se evalúa usando las muestras *out-of-bag*.
- ▶ Al final se obtiene una estimación promedio del desempeño:

$$\overline{\text{accuracy}} = \frac{1}{B} \sum_{b=1}^B \text{accuracy}_b$$

$$\overline{\text{loss}} = \frac{1}{B} \sum_{b=1}^B \text{loss}_b$$

*Esto permite estimar la variabilidad del modelo frente a diferentes muestras del dataset.*

## Ventajas y limitaciones del Bootstrap.

### Ventajas:

- ▶ Estima la varianza y sesgo de modelos.
- ▶ Útil incluso con datasets pequeños.
- ▶ **Aprovecha los datos varias veces**, generando distintos subconjuntos.

### Limitaciones:

- ▶ Mayor coste computacional: se entrena el modelo muchas veces.
- ▶ **Las muestras contienen datos repetidos, lo que puede generar sobreajuste** en algunos casos.
- ▶ No siempre es adecuado para datos secuenciales o dependientes temporalmente.



## Técnica Early Stopping

- ▶ Es una técnica de regularización que consiste en **detener el entrenamiento cuando el rendimiento en validación deja de mejorar**.
- ▶ Ayuda a evitar el sobreajuste, manteniendo el modelo en el punto donde generaliza mejor.

*Muy usada cuando se observa que la pérdida de validación empieza a aumentar mientras la de entrenamiento sigue bajando.*

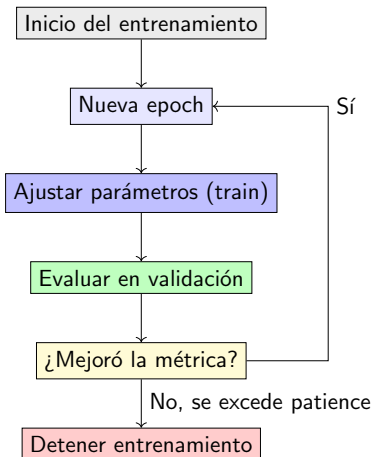
## Funcionamiento.

1. Durante el entrenamiento, se evalúa el modelo sobre el conjunto de validación cada cierta cantidad de **epochs**.
2. **Se guarda el modelo con la mejor métrica de validación** (por ejemplo, la menor  $\text{loss}_{val}$ ).
3. Si la métrica de validación no mejora tras un **número fijo de épocas** (*patience*), se detiene el entrenamiento.
4. Se **restaura el modelo al estado donde tuvo el mejor rendimiento** en validación.

$$\text{Best epoch} = \arg \min_e \text{loss}_{val}(e)$$

## Pipeline de entrenamiento con Early Stopping.

- Incluye monitoreo constante sobre el conjunto de validación:



## Ventajas y limitaciones de Early Stopping.

### Ventajas:

- ▶ **Reduce el sobreajuste** de forma automática.
- ▶ Reduce el tiempo de entrenamiento (se detiene antes).
- ▶ Sencillo de implementar y **muy usado en la práctica**.

### Limitaciones:

- ▶ Depende de elegir bien la métrica de validación.
- ▶ Sensible al valor de **patience** (puede detenerse muy pronto o demasiado tarde).
- ▶ No soluciona todos los problemas de sobreajuste, debe combinarse con otras técnicas (regularización, dropout, etc.).

## Métricas de Evaluación en Machine Learning.

En machine learning y deep learning, las métricas de evaluación permiten **medir cuantitativamente el desempeño** de los modelos.

- ▶ Ayudan a comparar distintos modelos y configuraciones.
- ▶ Guían el ajuste de hiperparámetros.
- ▶ Ayudan a detectar sobreajuste o subajuste.
- ▶ Se eligen según la tarea: clasificación, regresión, segmentación, etc.

## Clasificación Binaria.

En problemas de clasificación binaria, las métricas más comunes son:

- ▶ Exactitud (Accuracy)
- ▶ Precisión (Precision)
- ▶ Exhaustividad o Sensibilidad (Recall)
- ▶ Especificidad
- ▶ F1-score

Todas ellas derivan de la matriz de confusión.

## Matriz de Confusión.

La matriz de confusión compara las predicciones del modelo con las clases reales:

|               | Predicho Positivo       | Predicho Negativo       |
|---------------|-------------------------|-------------------------|
| Real Positivo | Verdadero Positivo (VP) | Falso Negativo (FN)     |
| Real Negativo | Falso Positivo (FP)     | Verdadero Negativo (VN) |

## Métricas:

- ▶ **Exactitud (Accuracy):** La exactitud mide la **proporción de predicciones correctas**:

$$\text{Accuracy} = \frac{VP + VN}{VP + FP + FN + VN}$$

Es útil cuando las clases están balanceadas, pero puede ser engañosa en casos desbalanceados.

- ▶ **Precisión (Precision):** proporción de verdaderos positivos entre las predicciones positivas.

$$\text{Precision} = \frac{VP}{VP + FP}$$

- ▶ **Exhaustividad (Recall / Sensibilidad):** proporción de verdaderos positivos entre los casos realmente positivos.

$$\text{Recall} = \frac{VP}{VP + FN}$$



- **Especificidad:** mide la proporción de verdaderos negativos correctamente clasificados.

$$\text{Specificity} = \frac{VN}{VN + FP}$$

- **F1-score:** media armónica entre precisión y recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Es útil cuando interesa equilibrar ambos.

## Ejemplo:

Se considera un modelo aplicado a 20 muestras:

- ▶ Casos reales positivos: 10
- ▶ Casos reales negativos: 10

Resultados de la predicción:

- ▶ 8 reales positivos fueron predichos como positivos (VP=8)
- ▶ 2 reales positivos fueron predichos como negativos (FN=2)
- ▶ 7 reales negativos fueron predichos como negativos (VN=7)
- ▶ 3 reales negativos fueron predichos como positivos (FP=3)

La matriz de confusión:

|               | Predicho Positivo | Predicho Negativo |
|---------------|-------------------|-------------------|
| Real Positivo | 8 (VP)            | 2 (FN)            |
| Real Negativo | 3 (FP)            | 7 (VN)            |

Cálculo de métricas a partir de la matriz de confusión:

- ▶ Accuracy:

$$\frac{VP + VN}{Total} = \frac{8 + 7}{20} = 0.75$$

- ▶ Precision:

$$\frac{VP}{VP + FP} = \frac{8}{8 + 3} \approx 0.727$$

- ▶ Recall (Sensibilidad):

$$\frac{VP}{VP + FN} = \frac{8}{8 + 2} = 0.8$$

- ▶ Specificity:

$$\frac{VN}{VN + FP} = \frac{7}{7 + 3} = 0.7$$

- ▶ F1-score:

$$2 \cdot \frac{0.727 \cdot 0.8}{0.727 + 0.8} \approx 0.761$$

## Matriz de Confusión en Problemas Multiclase.

En problemas multiclase (con  $K$  clases), la matriz de confusión permite resumir el rendimiento del modelo:

- ▶ Se crea una matriz cuadrada de tamaño  $K \times K$ .
- ▶ Las filas representan las clases reales y las columnas las clases predichas.
- ▶ Para cada muestra del conjunto de prueba:
  - ▶ Se identifica la clase real  $C_i$ .
  - ▶ Se identifica la clase predicha  $C_j$ .
  - ▶ Se incrementa en 1 el valor de la celda  $M_{ij}$ .
- ▶ Al finalizar, cada celda  $M_{ij}$  indica cuántas veces la clase real  $C_i$  fue predicha como  $C_j$ .

Los elementos diagonales ( $M_{11}, M_{22}, \dots, M_{KK}$ ) corresponden a las predicciones correctas.

**Matriz de confusión general para un problema multiclase con  $K$  clases.**

| Real     | Predicha              |          |                       | Total                 |
|----------|-----------------------|----------|-----------------------|-----------------------|
|          | $C_1$                 | $\dots$  | $C_K$                 |                       |
| $C_1$    | $M_{11}$              | $\dots$  | $M_{1K}$              | $\sum_{j=1}^K M_{1j}$ |
| $\vdots$ | $\vdots$              | $\ddots$ | $\vdots$              | $\vdots$              |
| $C_K$    | $M_{K1}$              | $\dots$  | $M_{KK}$              | $\sum_{j=1}^K M_{Kj}$ |
| Total    | $\sum_{i=1}^K M_{i1}$ | $\dots$  | $\sum_{i=1}^K M_{iK}$ | $N$                   |

Cada celda  $M_{ij}$  indica cuántas veces la clase real  $C_i$  fue predicha como  $C_j$ .

**Ejemplo Numérico:** Tras evaluar el modelo con 30 imágenes se obtuvo la siguiente matriz:

|              | Predicho<br>Perro | Predicho<br>Gato | Predicho<br>Conejo | Predicho<br>Hámster |
|--------------|-------------------|------------------|--------------------|---------------------|
| Real Perro   | 6                 | 1                | 0                  | 0                   |
| Real Gato    | 2                 | 5                | 1                  | 0                   |
| Real Conejo  | 0                 | 1                | 4                  | 1                   |
| Real Hámster | 0                 | 0                | 2                  | 7                   |

## Interpretación:

- ▶ El modelo clasificó correctamente: 6 perros, 5 gatos, 4 conejos y 7 hámsters.
- ▶ Hubo confusiones frecuentes: 2 gatos fueron predichos como perros, y 2 hámsters fueron confundidos como conejos.

## Matriz de confusión para Clase “Perro”.

|              | Predicho Perro | Predicho Gato | Predicho Conejo | Predicho Hámster |
|--------------|----------------|---------------|-----------------|------------------|
| Real Perro   | 6              | 1             | 0               | 0                |
| Real Gato    | 2              | 5             | 1               | 0                |
| Real Conejo  | 0              | 1             | 4               | 1                |
| Real Hámster | 0              | 0             | 2               | 7                |

**Legenda:**  VP  FP  FN  VN

## Cálculo de Métricas para Clase “Perro”

$$TP = 6, \quad FP = 2, \quad FN = 1, \quad TN = 21$$

$$\text{Precision} = \frac{VP}{VP + FP} = \frac{6}{6 + 2} = 0.75, \quad \text{Recall} = \frac{VP}{VP + FN} = \frac{6}{6 + 1} = 0.857$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 0.8, \quad \text{Specificity} = \frac{VN}{VN + FP} = \frac{21}{21 + 2} = 0.913$$

## Matriz de confusión para Clase “Gato”.

|              | Predicho<br>Perro | Predicho<br>Gato | Predicho<br>Conejo | Predicho<br>Hámster |
|--------------|-------------------|------------------|--------------------|---------------------|
| Real Perro   | 6                 | 1                | 0                  | 0                   |
| Real Gato    | 2                 | 5                | 1                  | 0                   |
| Real Conejo  | 0                 | 1                | 4                  | 1                   |
| Real Hámster | 0                 | 0                | 2                  | 7                   |

**Leyenda:** ■ VP ■ FP ■ FN ■ VN

## Cálculo de Métricas para Clase “Gato”

$$TP = 5, \quad FP = 2, \quad FN = 3, \quad TN = 20$$

$$\text{Precision} = \frac{5}{5 + 2} = 0.714, \quad \text{Recall} = \frac{5}{5 + 3} = 0.625$$

$$F1 = 0.667, \quad \text{Specificity} = \frac{20}{20 + 2} = 0.909$$



## Cálculo de Métricas para Clase Conejo

$$TP = 4, \quad FP = 3, \quad FN = 2, \quad TN = 21$$

$$\text{Precision} = \frac{4}{4 + 3} = 0.571, \quad \text{Recall} = \frac{4}{4 + 2} = 0.667$$

$$F1 = 0.614, \quad \text{Specificity} = \frac{21}{21 + 3} = 0.875$$

## Cálculo de Métricas para Clase Hámster

$$TP = 7, \quad FP = 1, \quad FN = 2, \quad TN = 20$$

$$\text{Precision} = \frac{7}{7 + 1} = 0.875, \quad \text{Recall} = \frac{7}{7 + 2} = 0.778$$

$$F1 = 0.823, \quad \text{Specificity} = \frac{20}{20 + 1} = 0.952$$

## Resumen de Métricas y Macro-Promedios.

**Accuracy:**

$$\frac{6 + 5 + 4 + 7}{30} = \frac{22}{30} \approx 0.733$$

**Macro-promedio:**

$$\text{Precision macro} = \frac{0.75 + 0.714 + 0.571 + 0.875}{4} \approx 0.728$$

$$\text{Recall macro} = \frac{0.857 + 0.625 + 0.667 + 0.778}{4} \approx 0.732$$

$$\text{F1 macro} = \frac{0.8 + 0.667 + 0.614 + 0.823}{4} \approx 0.726$$

$$\text{Specificity macro} = \frac{0.913 + 0.909 + 0.875 + 0.952}{4} \approx 0.912$$

**Ver código python.**

## Curva ROC.

- ▶ La curva ROC (Receiver Operating Characteristic) muestra la relación entre:
  - ▶ **TPR (True Positive Rate):**

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- ▶ **FPR (False Positive Rate):**

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

- ▶ Se construye **variando el umbral de decisión** del modelo.
- ▶ Cada punto en la curva corresponde a un par (FPR, TPR) para un determinado umbral.

## El umbral (threshold):

- ▶ Es el valor a partir del cual se decide si una muestra se clasifica como positiva.
- ▶ Por defecto, suele ser 0.5 (en problemas balanceados).
- ▶ En clasificación también se llama **frontera de decisión** o **punto de corte**.

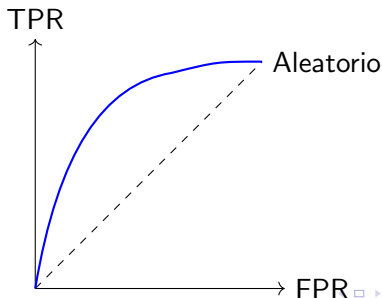
## Para construir la curva ROC:

- ▶ Se hace **variar el umbral** de decisión desde valores muy bajos (clasifica todo como positivo) hasta muy altos (clasifica todo como negativo).
- ▶ En cada paso, se recalculan las etiquetas predichas, la matriz de confusión y se obtiene un nuevo par (FPR, TPR).

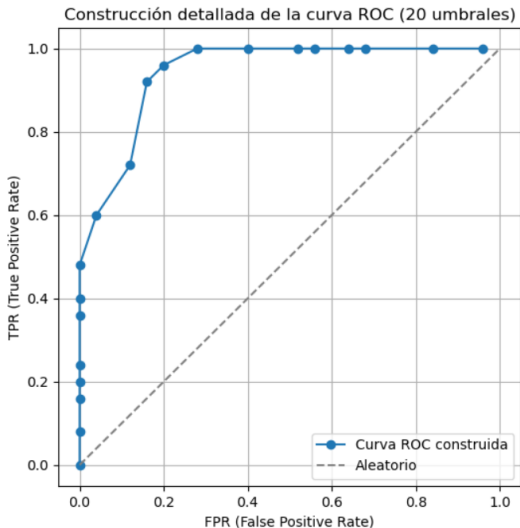
Así se explora el comportamiento del modelo en distintos escenarios, observando el trade-off entre sensibilidad (TPR) y especificidad ( $1 - \text{FPR}$ ).

## Interpretación de la Curva ROC.

- ▶ La diagonal ( $y = x$ ) representa el comportamiento de un clasificador aleatorio.
- ▶ Cuanto más se acerque la curva ROC a la esquina superior izquierda (TPR=1, FPR=0), mejor será el rendimiento del modelo.
- ▶ La curva ROC permite comparar diferentes clasificadores, observando cuál domina (está más arriba a la izquierda).



## Ejemplo.

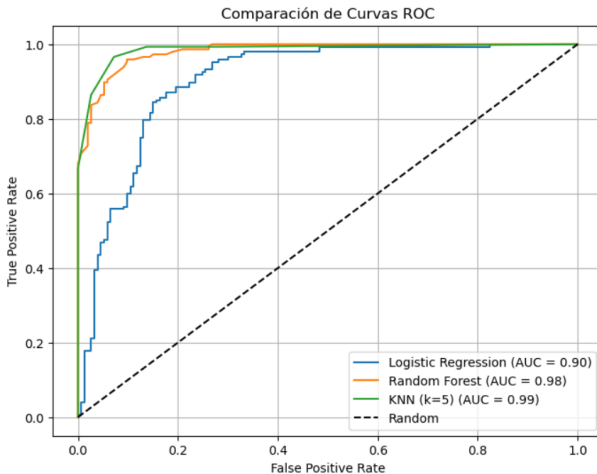


## AUC: Área Bajo la Curva ROC.

- ▶ El **AUC** (Area Under the Curve) es el área bajo la curva ROC.
- ▶ Valores típicos:
  - ▶  $AUC = 0.5$ : modelo no mejor que el azar.
  - ▶  $AUC = 1$ : modelo perfecto.
- ▶ Se interpreta como la probabilidad de que el modelo asigne una mayor puntuación a una muestra positiva que a una negativa, elegidas al azar.

$$AUC = \int_0^1 TPR(FPR) d(FPR)$$

## Ejemplo.



**Ver código python.**



## Ventajas de la curva ROC.

- ▶ Permite evaluar el rendimiento de un modelo sin depender de un único umbral.
- ▶ Útil especialmente cuando las **clases** están **desbalanceadas**.
- ▶ Facilita la **comparación entre distintos modelos**.
- ▶ Ayuda a **seleccionar el umbral** que mejor se adapte a las necesidades del problema.

## Selección del umbral usando la curva ROC.

La curva ROC permite visualizar el comportamiento del modelo para todos los posibles umbrales de decisión.

Así se puede:

- ▶ Elegir un umbral que maximice la sensibilidad (TPR) si el problema requiere detectar la mayor cantidad de positivos (por ejemplo, en diagnóstico médico).
- ▶ O elegir un umbral que minimice la FPR si es más crítico evitar falsos positivos (por ejemplo, en sistemas de control de calidad).

En resumen, la curva ROC no sólo evalúa el modelo, sino que también ayuda a seleccionar el umbral que mejor equilibre los errores según el contexto del problema.

## Cómo encontrar el umbral óptimo.

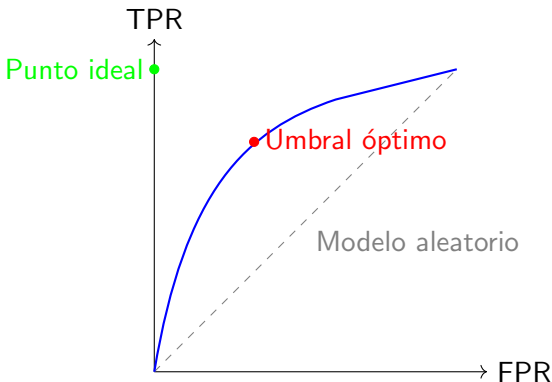
Para seleccionar el mejor umbral:

1. Construir la curva ROC: cada punto corresponde a un umbral concreto.
2. Elegir el umbral que optimice algún criterio según el problema.

## Criterios más comunes:

| Criterio                  | Cálculo                                          | Idea principal                                                                 |
|---------------------------|--------------------------------------------------|--------------------------------------------------------------------------------|
| Mayor Youden's J          | $J = \text{TPR} - \text{FPR}$                    | Máxima diferencia entre TPR y FPR; balance entre sensibilidad y especificidad. |
| Punto más cercano a (0,1) | $d = \sqrt{(\text{FPR})^2 + (1 - \text{TPR})^2}$ | Mejor equilibrio, minimizando distancia al vértice ideal.                      |
| Ajuste según prioridad    | –                                                | Según si se prefiere más sensibilidad o más especificidad.                     |

## Cómo encontrar el umbral óptimo.



Este punto óptimo puede encontrarse:

- ▶ Como el que maximiza  $TPR - FPR$  (Youden's J).
- ▶ O el más cercano al vértice superior izquierdo (0,1).

## Métricas de regresión

En problemas de regresión, las métricas de evaluación cuantifican:

- ▶ La **magnitud media del error** entre las predicciones y los valores reales.
- ▶ El impacto que tienen los errores grandes.
- ▶ La capacidad explicativa del modelo respecto a la **variabilidad de los datos**.
- ▶ Si existe **sesgo o tendencia** sistemática en las predicciones.

No buscan “aciertos o errores exactos” (como en clasificación), sino medir **qué tan lejos** están las predicciones del valor real.

## Mean Absolute Error.

Promedio del valor absoluto de los errores entre las predicciones y los valores reales.

### Fórmula:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

### Interpretación:

- ▶ Indica, en promedio, cuántas unidades se equivoca el modelo.
- ▶ Es intuitivo y fácil de explicar, ya que se expresa en la misma escala que la variable predicha.

## Mean Squared Error.

Promedio de los cuadrados de los errores; penaliza más los errores grandes.

### Fórmula:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

### Interpretación:

- ▶ Penaliza más los errores grandes debido a la elevación al cuadrado.
- ▶ Útil cuando se busca que el modelo evite errores muy grandes.

## Root Mean Squared Error.

Raíz cuadrada del MSE, expresada en la misma escala que la variable predicha.

### Fórmula:

$$RMSE = \sqrt{MSE}$$

### Interpretación:

- ▶ Similar al MSE pero más interpretable por estar en la **misma unidad que los datos**.
- ▶ También penaliza más los errores grandes.



## $R^2$ (Coeficiente de determinación)

Proporción de la varianza de los datos que es explicada por el modelo.

### Fórmula:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

donde  $\bar{y}$  es el promedio de los valores reales.

### Interpretación:

- ▶ Indica qué proporción de la variabilidad de los datos se explica gracias al modelo.
- ▶ **Idealmente cercano a 1;**  
puede ser negativo si el modelo es peor que simplemente predecir el promedio.

## MAPE (Mean Absolute Percentage Error).

Error medio expresado como porcentaje del valor real;  
útil para interpretar errores en términos relativos.

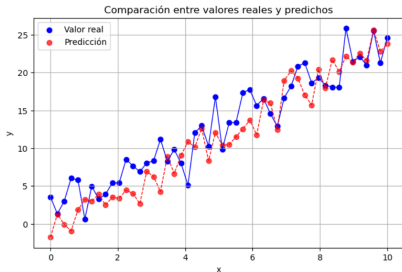
### Fórmula:

$$MAPE = \frac{100\%}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

### Interpretación:

- ▶ Permite saber, en promedio, qué porcentaje de error tiene el modelo.
- ▶ Se debe usar con cuidado si algunos valores reales  $y_i$  son cercanos a cero, pues distorsiona el resultado.

## Ejemplo:



- ▶ **MAE:**  $\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| = 2.38$
- ▶ **MSE:**  $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = 8.83$
- ▶ **RMSE:**  $\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} = 2.97$
- ▶  **$R^2$ :**  $1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 0.812$
- ▶ **MAPE:**  $\frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| = 36.13$

**Ver código python.**

## Métricas para segmentación

En segmentación de imágenes, el objetivo es evaluar qué tan bien las regiones predichas coinciden con la verdad de terreno (**ground truth**).

Estas métricas cuantifican:

- ▶ La cantidad de píxeles correctamente clasificados como positivos o negativos.
- ▶ La coincidencia entre las áreas predichas y las reales.
- ▶ La calidad y cobertura de la segmentación, especialmente en objetos pequeños.

Las métricas se derivan de la matriz de confusión de píxeles: VP, FP, FN y VN.

En segmentación también se aplican métricas clásicas de clasificación:

- **Precisión (Precision).**

$$\text{Precisión} = \frac{VP}{VP + FP}$$

Proporción de píxeles predichos como positivos que realmente son positivos.

- **Sensibilidad (Recall).**

$$\text{Sensibilidad} = \frac{VP}{VP + FN}$$

Proporción de píxeles positivos correctamente identificados.

- **Exactitud (Accuracy).**

$$\text{Accuracy} = \frac{VP + VN}{\text{Total de píxeles}}$$

Proporción total de píxeles correctamente clasificados.

IoU (Intersection over Union).

Mide la proporción entre el área de intersección y el área de unión entre la máscara predicha y la **Ground Truth** (máscara verdadera).

**Fórmula:**

$$IoU = \frac{|Pred \cap GT|}{|Pred \cup GT|}$$

**Interpretación:**

- ▶ **Valor entre 0 y 1; cuanto mayor, mejor.**
- ▶ También conocido como **Jaccard Index**.
- ▶ Evalúa qué tanto se solapan las áreas predicha y real.

Dice coefficient (F1 score).

Métrica de similitud que pondera el doble la intersección; más sensible a errores en objetos pequeños.

**Fórmula:**

$$Dice = \frac{2 \cdot |Pred \cap GT|}{|Pred| + |GT|}$$

**Interpretación:**

- ▶ **Valor entre 0 y 1; cuanto mayor, mejor.**
- ▶ Equivalente al F1-score en clasificación.
- ▶ Suele dar resultados ligeramente más altos que IoU, especialmente cuando los objetos son pequeños.

## Comparación entre IoU y Dice coefficient.

### Similitudes:

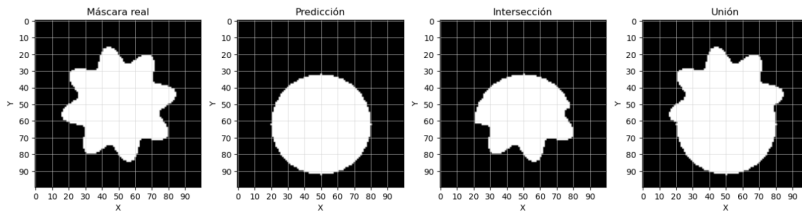
- ▶ Ambas miden la superposición entre la máscara predicha y la verdad de terreno.
- ▶ Sus valores están acotados entre 0 y 1.
- ▶ Cuanto más altos, mejor rendimiento del modelo.

### Diferencias:

- ▶ **Dice** pondera el doble la intersección, siendo más tolerante en casos con objetos pequeños.
- ▶ **IoU** tiende a valores ligeramente menores que Dice.
- ▶ Interpretación:  $\text{Dice} \approx \frac{2 \cdot \text{IoU}}{1 + \text{IoU}}$ .



## Diagrama ilustrativo.



- ▶ IoU (Intersection over Union): 0.586
- ▶ Dice Coefficient: 0.739

## Ejemplo numérico de IoU y Dice.

Supóngase una segmentación binaria con:

- ▶  $|\text{Pred}| = 70$  píxeles positivos predichos.
- ▶  $|\text{GT}| = 60$  píxeles reales positivos.
- ▶ Intersección:  $|\text{Pred} \cap \text{GT}| = 50$  píxeles.

**Cálculos:**

$$\text{IoU} = \frac{50}{70 + 60 - 50} = \frac{50}{80} = 0.625$$

$$\text{Dice} = \frac{2 \cdot 50}{70 + 60} = \frac{100}{130} \approx 0.769$$

**Interpretación:**

- ▶  $\text{IoU} \approx 62.5\%$  de solapamiento.
- ▶  $\text{Dice} \approx 76.9\%$ ; refleja mayor sensibilidad a la intersección.

**Ver código python.**

## Consideraciones al elegir métricas

- ▶ Dependen del **tipo de problema** (clasificación, regresión, segmentación, etc.).
- ▶ Se deben tener en cuenta desequilibrios en las clases.
- ▶ Las métricas pueden complementarse para obtener una visión más completa.
- ▶ Es recomendable validar usando técnicas como cross-validation.

## Conclusión.

Las métricas de evaluación permiten:

- ▶ Medir el rendimiento real de un modelo.
- ▶ Comparar distintas configuraciones y algoritmos.
- ▶ Tomar decisiones fundamentadas durante el ajuste y despliegue.

La correcta selección de métricas es clave para desarrollar modelos robustos.

## Desbalance de Datos (clases).

En muchos problemas de clasificación, una o más clases pueden tener muchas menos muestras que el resto.

Este fenómeno se conoce como **Desbalance de Datos**.

Ejemplos:

- ▶ Detección de fraudes: muy pocas transacciones fraudulentas frente a millones de transacciones legítimas.
- ▶ Diagnóstico médico: pocas imágenes con enfermedad frente a muchas normales.

## Por qué es un problema.

- ▶ Los modelos tienden a **priorizar la clase mayoritaria**, ignorando las clases minoritarias.
- ▶ Puede dar lugar a una **exactitud (accuracy) engañosamente alta**.
- ▶ Las clases minoritarias suelen ser precisamente las más importantes (ej. fraude, enfermedad).

Por ello, se necesitan técnicas que permitan equilibrar el aprendizaje del modelo.

## Técnicas para mitigar el desbalance.

Las técnicas se pueden clasificar en:

- ▶ **Modificación de los datos** de entrada (submuestreo, sobremuestreo, generación sintética).
- ▶ **Ajuste del algoritmo** de aprendizaje (ponderación de clases, uso de métricas robustas).

Cada estrategia tiene ventajas y limitaciones según el problema y los datos.

## Submuestreo (undersampling).

Reducir el número de muestras de la clase mayoritaria para equilibrar el dataset.

### **Ventajas:**

- ▶ Simple de implementar.
- ▶ Reduce tiempo de entrenamiento.

### **Desventajas:**

- ▶ Puede perder información relevante de la clase mayoritaria.



## Sobremuestreo (oversampling).

Replicar o crear copias adicionales de muestras de la clase minoritaria.

### **Ventajas:**

- ▶ Mantiene toda la información de la clase mayoritaria.

### **Desventajas:**

- ▶ Riesgo de sobreajuste por repetición de datos idénticos.

## Generación sintética (SMOTE).

Crear **nuevas muestras sintéticas** de la clase minoritaria interpolando entre ejemplos existentes.

### Ventajas:

- ▶ Reduce el sobreajuste frente al simple sobremuestreo.
- ▶ Mejora la cobertura de la clase minoritaria.

### Desventajas:

- ▶ Puede generar muestras no realistas si los datos son muy dispersos.

## Ponderación de clases

Ajustar el algoritmo para que penalice más los errores cometidos sobre la clase minoritaria.

## Métricas robustas

Usar métricas que no dependan del balance de clases, como:

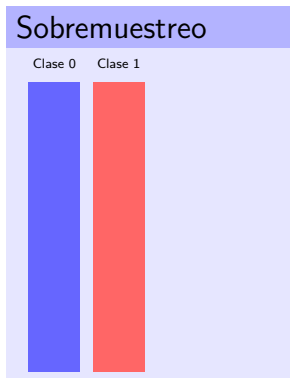
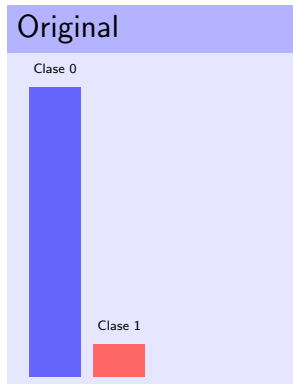
- ▶ F1-score
- ▶ AUC-ROC
- ▶ Balanced accuracy

Estas estrategias ayudan a evaluar y entrenar modelos que consideren correctamente las clases menos representadas.

# Desbalance de Datos

**Ejemplo visual:** Sea un dataset de clasificación binaria:

- ▶ Clase 0 (mayoritaria): 900 muestras
- ▶ Clase 1 (minoritaria): 100 muestras



Visualmente se aprecia cómo cambian las proporciones de las clases al aplicar distintas técnicas de balanceo.

## Data augmentation.

Data augmentation es un conjunto de técnicas que **generan nuevas muestras artificiales** a partir de las existentes, aplicando transformaciones que mantienen su clase.

### Ejemplos más comunes:

- ▶ Rotaciones, traslaciones o escalados en imágenes.
- ▶ Cambios de brillo, contraste o recorte.
- ▶ Sinónimos o reordenamiento de palabras en texto.

### Relación con el desbalance:

- ▶ Aunque su objetivo principal es mejorar la generalización del modelo, también puede aplicarse sólo sobre la clase minoritaria, actuando como técnica de sobremuestreo.

## Similitudes entre SMOTE y data augmentation.

Ambas técnicas comparten objetivos:

- ▶ Generar muestras adicionales para enriquecer el conjunto de entrenamiento.
- ▶ Ayudar a que el modelo generalice mejor y reducir el sobreajuste.
- ▶ Son especialmente útiles cuando se dispone de pocas muestras reales.

Sin embargo, difieren en el tipo de datos y la forma de generar las muestras.

## Diferencias clave entre SMOTE y data augmentation.

| Aspecto             | SMOTE                                                       | Data augmentation                                                        |
|---------------------|-------------------------------------------------------------|--------------------------------------------------------------------------|
| Tipo de datos       | Datos tabulares o numéricos                                 | Imágenes, texto, audio                                                   |
| Cómo se crean       | Interpolación entre muestras reales de la clase minoritaria | Transformaciones sobre datos existentes (rotar, escalar, recortar, etc.) |
| Objetivo principal  | Balancear clases (crear más muestras minoritarias)          | Incrementar la variabilidad y robustez del modelo                        |
| Forma de generación | Basada en el espacio de características                     | Basada en el dominio original de los datos                               |

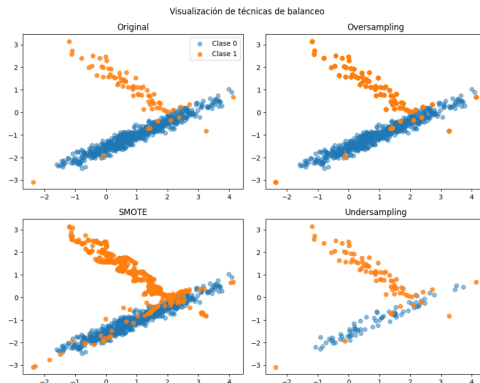
## Ejemplo.

- ▶ **SMOTE**: en un dataset financiero, si solo hay 100 casos fraudulentos, SMOTE genera nuevas transacciones sintéticas interpolando características de casos reales.
- ▶ **Data augmentation**: en un conjunto de imágenes de tumores, se aplican rotaciones, traslaciones o cambios de brillo para aumentar la diversidad de imágenes que el modelo ve.

En ambos casos se generan más datos, pero la técnica y el propósito específico varían.



## Ejemplo.



| Técnica       | Accuracy | F1-score |
|---------------|----------|----------|
| Original      | 0.973333 | 0.851852 |
| Oversampling  | 0.963333 | 0.835821 |
| SMOTE         | 0.960000 | 0.823529 |
| Undersampling | 0.973333 | 0.862069 |
| Weighted LR   | 0.976667 | 0.888889 |

Ver código python.

## Conclusiones.

- ▶ El Desbalance de Datos puede afectar gravemente el desempeño real de un modelo, aunque la **exactitud** aparente sea alta.
- ▶ Existen técnicas basadas en los datos:
  - ▶ **Submuestreo**: reducir muestras de la clase mayoritaria.
  - ▶ **Sobremuestreo**: replicar o generar más muestras de la clase minoritaria.
  - ▶ **SMOTE**: generar ejemplos sintéticos mediante interpolación.
  - ▶ **Data augmentation**: transformar datos existentes, que puede aplicarse para mejorar la robustez o balancear clases minoritarias.
- ▶ También se pueden ajustar los algoritmos:
  - ▶ Ponderar más los errores sobre la clase minoritaria.
  - ▶ Usar métricas que sean menos sensibles al desbalance (F1-score, AUC-ROC).
- ▶ La combinación de varias estrategias suele ofrecer los mejores resultados.

**Gracias por su atención.**

